ASSEMBLY LANGUAGE

THE LANGUAGE OF MACHINES

Basics of Assembly Language

A COMPREHENSIVE GUIDE TO 8086 ASSEMBLY PROGRAMMING

The 8086 Microprocessor

The Intel 8086 is a **16-bit microprocessor** introduced in **1978** that became the foundation of the x86 architecture family. It revolutionized computing by enabling more powerful software and complex operations.

Key Specifications

16-bit Data Bus Width	**20-bit** Address Bus
1 MB Memory Addressable	**5-10 MHz** Clock Speed

Why Learn 8086?

1. **Foundation:** Understanding 8086 provides the basis for all x86 processors
2. **Simplicity:** 16-bit architecture is easier to learn than modern 64-bit
3. **Concepts:** Core concepts apply to all assembly languages
4. **Tools:** Excellent emulators like **emu8086** available
5. **Legacy:** Still relevant for embedded and legacy systems

Historical Impact: The 8086's architecture lives on in every modern Intel and AMD processor!

02

Assembly Program Structure

UNDERSTANDING THE THREE ESSENTIAL SECTIONS OF AN ASSEMBLY PROGRAM

The Three Sections Overview

An assembly program is organized into **three main sections**, each serving a distinct purpose. Understanding these sections is essential for writing proper assembly code.

DATA

Initialized Data Section

Purpose: Declare constants and initialized data

Used for values that don't change at runtime

Syntax:

```
section .data
```

BSS

Uninitialized Data Section

Purpose: Declare variables

Reserves space for data that will be modified

Syntax:

```
section .bss
```

TEXT

Code Section

Purpose: Contains executable code

Where the actual program instructions reside

Syntax:

```
section .text
```

Program Flow: The assembler processes each section in order, converting your code into machine language that the CPU can execute.

Data Section Explained

The **data section** is used for declaring initialized data or constants. This data does not change at runtime and remains fixed throughout program execution.

Common Use Cases

1. **Constant Values**
   Mathematical constants, configuration values
2. **String Messages**
   Text to display to users
3. **File Names**
   Paths and file identifiers
4. **Lookup Tables**
   Pre-calculated values for fast access

Syntax & Examples

```
; Declare the section
section .data
```

```
; Define constants
MAX_SIZE dw 100
PI dd 3.14159
```

```
; Define strings
message db 'Hello, World!'
filename db 'data.txt', 0
```

```
; Define arrays
numbers db 1, 2, 3, 4, 5
```

Key Point: Data in this section is stored in the executable file and loaded into memory when the program starts.

BSS Section Explained

The **bss section** (Block Started by Symbol) is used for declaring variables. Unlike the data section, variables here are uninitialized and will be set to zero by default.

Key Characteristics

- **Uninitialized Storage**
  Reserves memory without initial values
- **Zero-Initialized**
  Automatically set to zero at program start
- **Modifiable at Runtime**
  Values can change during execution
- **Smaller Executable**
  Doesn't increase file size like .data

Syntax & Examples

```
; Declare the section
section .bss
```

```
; Reserve single byte
counter resb 1
```

```
; Reserve word (2 bytes)
total resw 1
```

```
; Reserve double word (4 bytes)
result resd 1
```

```
; Reserve array of 10 bytes
buffer resb 10
```

Best Practice: Use .bss for variables that start at zero to make your executable smaller and more efficient.

Text Section Explained

The **text section** contains the actual executable code of your program. This is where you write the instructions that the CPU will execute.

Required Elements

global _start

Tells the kernel where program execution begins. This declaration is mandatory.

_start: Label

Marks the entry point where the CPU begins executing instructions.

Complete Example

```
section .data
    msg db 'Hello', 0

section .bss
    buffer resb 100

section .text
    global _start
    _start:
        mov ax, 5
        add ax, 3
        ; ... more code
```

Important: Without global _start , the kernel won't know where to begin execution!

What is the Kernel?

The kernel is the central component of an operating system that manages computer hardware and allows software programs to interact with it

Comments in Assembly

In assembly language, **comments begin with a semicolon (;)**. Everything after the semicolon on that line is ignored by the assembler.

Why Comments Matter

Documentation
Explain what the code does for future readers

Debugging
Help identify issues and track logic flow

Collaboration
Help team members understand your code

Learning
Essential for students and beginners

Comment Examples

```
; This is a full-line comment
mov ax, 5
```

```
mov bx, 10 ; Load counter value
```

```
; Calculate sum of two numbers
add ax, bx ; AX = AX + BX
```

```
;;;
```

Types of Assembly Statements

Assembly language programs consist of **three types of statements**, each serving a different purpose in the program.

Executable Instructions

Tell the processor what to do

Direct commands that the CPU executes

Examples:

MOV, ADD, SUB
JMP, CMP, INC

- ✓ Generate machine code
- ✓ Change register values
- ✓ Modify memory

Assembler Directives

Guide the assembly process

Instructions for the assembler itself

Examples:

```
section .data
global _start
```

- ✓ Non-executable
- ✓ Don't generate machine code
- ✓ Organize program structure

Macros

Text substitution mechanism

Reusable code sequences with names

Example:

```
%macro print 1
    mov dx, %1
    int 21h
%endmacro
```

- ✓ Code reuse
- ✓ Simplify repetitive tasks
- ✓ Expand at assembly time

Remember: Only executable instructions become part of the final machine code. Directives and macros help organize and simplify the source code.

Executable Instructions

Executable instructions tell the processor what to do. Each instruction consists of an operation code (opcode) and generates one machine language instruction.

Instruction Components

Operation Code (Opcode)

The specific operation to perform (e.g., MOV, ADD)

Operands

Data or locations the operation works on

Common Instructions

MOV Move data	mov ax, bx
ADD Addition	add ax, 5
SUB Subtraction	sub bx, cx
INC Increment	inc ax
CMP Compare	cmp ax, bx

Execution: Each instruction is fetched, decoded, and executed by the CPU in sequence.

Assembler Directives

Assembler directives (pseudo-ops) tell the assembler about various aspects of the assembly process. These are non-executable and do not generate machine language instructions.

Key Characteristics

For the Assembler
Instructions to the assembler tool, not the CPU

Non-Executable
Don't become part of the machine code

Organize Code
Define sections, reserve memory, declare symbols

Common Directives

section
Define program sections (.data, .bss, .text)

```
section .data
```

global
Make symbol visible to linker

```
global _start
```

db, dw, dd
Define bytes, words, double words

```
value db 100
```

resb, resw, resd
Reserve storage space

```
buffer resb 256
```

equ
Define constants

```
MAX_SIZE equ 100
```

Macros in Assembly

Macros are a text substitution mechanism. A macro is a sequence of instructions assigned a name that can be used anywhere in the program.

Benefits of Macros

Code Reuse
Write once, use multiple times

Reduce Code Size
Shorter, more readable source files

Easy Maintenance
Change in one place, applies everywhere

Improved Readability
Abstract complex operations

Macro Example

```
; Define a macro to print character
%macro print_char 1
    mov ah, 02h ; Function to print
    mov dl, %1 ; Character to print
    int 21h ; Call DOS interrupt
%endmacro
```

```
; Use the macro
print_char 'A' ; Prints 'A'
print_char 'B' ; Prints 'B'
```

```
; Macro with multiple parameters
%macro add_numbers 2
    mov ax, %1
    add ax, %2
%endmacro
```

How It Works: The assembler replaces macro calls with the actual code during assembly time (text substitution).

Assembly Statement Syntax

Assembly language statements are entered **one statement per line**. Each statement follows a specific format with optional and required fields.

Statement Format

```
[ label ]  [ mnemonic ]  [ operands ]  [ ;comment ]
```

1. **Label (Optional)**
   Identifier for the statement, used for referencing
2. **Mnemonic (Required)**
   The instruction name (MOV, ADD, SUB, etc.)
3. **Operands (Optional)**
   Data or locations the instruction works on
4. **Comment (Optional)**
   Documentation starting with semicolon

Syntax Examples

```
; Full statement with all fields
start: mov ax, 5 ; Initialize

; No label
add ax, bx

; Just a label
loop_start:

; Instruction with comment
inc cx ; Counter++

; Just a comment
; End of program
```

Note: Fields in brackets [ ] are optional. The mnemonic is the only required field for executable instructions.

Instruction Components

A basic instruction has **two main parts**: the mnemonic (instruction name) and the operands (parameters).

1 Mnemonic

The name of the instruction to be executed. It tells the CPU what operation to perform.

Examples:

- MOV — Move data
- ADD — Addition
- SUB — Subtract
- JMP — Jump

2 Operands

The parameters or data that the instruction works on. Can be registers, memory locations, or immediate values.

Types:

- Registers: AX, BX, CX
- Memory: [1234h], [bx]
- Immediate: 5, 0FFh, 'A'

Visual Breakdown

MOV → AX , 5

Mnemonic Destination Source

Common Instruction Examples

INC Increment

```
INC COUNT
```

Operation: Increment the memory variable COUNT by 1

Equivalent to: COUNT = COUNT + 1

MOV Move

```
MOV TOTAL, 48
```

Operation: Transfer the value 48 to memory variable TOTAL

Equivalent to: TOTAL = 48

ADD Addition

```
ADD AH, BH
```

Operation: Add contents of BH register to AH register

Equivalent to: AH = AH + BH

AND Logical AND

```
AND MASK1, 128
```

Operation: Perform bitwise AND on MASK1 and 128

Equivalent to: MASK1 = MASK1 & 128

ADD Add Immediate

```
ADD MARKS, 10
```

Operation: Add 10 to the variable MARKS

Equivalent to: MARKS = MARKS + 10

MOV Move to Register

```
MOV AL, 10
```

Operation: Transfer value 10 to AL register

Equivalent to: AL = 10

🔷 Example

Say you write this simple assembly code for a made-up CPU:

```
MOV A, 5     ; put 5 in register A
ADD A, 3     ; add 3 to A
```

A CPU emulator would keep track of `A` and update it:

- After first instruction → `A = 5`
- After second instruction → `A = 8`

That's exactly what real hardware would do.

03

emu8086 Emulator

LEARNING ASSEMBLY WITHOUT PHYSICAL HARDWARE

What is an Emulator?

emu8086 is an emulator (not a real processor, but a software simulation) for the Intel 8086 microprocessor. It lets you write, compile and run assembly language programs on your computer without needing the actual old hardware.

What is an Emulator?

Software Simulation
A program that mimics how a CPU runs assembly instructions

Instruction Set Support
Understands the same commands as the real processor

Step-by-Step Execution
Executes each instruction just like the real CPU would

Why Use emu8086?

Learn Without Hardware
No need for vintage computers or physical processors

Visual Debugging
See registers, memory, and flags change in real-time

Safe Environment
Can't damage your computer while learning

Instant Feedback
Write, compile, and run code immediately

Popular Choice: emu8086 is widely used for learning x86 assembly programming because it provides a simple interface and powerful visual tools.

How Emulators Work

Every CPU (like Intel x86, ARM, MIPS, etc.) has its own **instruction set** — the list of assembly commands it understands, like MOV, ADD, JMP. An emulator "pretends" to be that CPU.

1

Fetch

The emulator reads the next instruction from memory

2

Decode

The instruction is interpreted and understood

3

Execute

The operation is performed on virtual registers/memory

Emulation Benefits

- **Practice Without Hardware**
  Learn assembly without needing physical vintage computers
- **Step-by-Step Debugging**
  Execute one instruction at a time and observe changes
- **Visual State Display**
  See registers, flags, and memory in real-time

What emu8086 Simulates

- CPU Registers ✓
- Memory (RAM) ✓
- Stack Operations ✓
- FLAGS Register ✓
- I/O Ports ✓

Benefits of Using emu8086

Built-in Editor

Write assembly code directly in the emulator with syntax highlighting and error detection.

No need for separate text editors or IDEs.

Compile & Run

Compile and test code immediately with a single click. Instant feedback on errors.

Fast iteration cycle for learning and debugging.

Register Display

Visual representation of all registers: AX, BX, CX, DX, SI, DI, SP, BP, IP, FLAGS.

Watch values change as you step through code.

Memory View

See stack, memory, and I/O ports in real-time. Understand how data is stored.

Hex dump view with ASCII representation.

Simulated Peripherals

Practice I/O operations with simulated screen, keyboard, and other devices.

Learn real-world programming scenarios.

Beginner-Friendly

Designed specifically for learning with clear interface and helpful features.

Perfect for students and self-learners.

emu8086 Features Overview

emu8086 provides a comprehensive environment for learning and practicing 8086 assembly programming with powerful visual tools.

Code Editor

- ✓ Syntax highlighting
- ✓ Error detection
- ✓ Line numbers
- ✓ Auto-indentation

Register Window

- ✓ All 16-bit registers
- ✓ 8-bit sub-registers
- ✓ Segment registers
- ✓ Pointer/index registers

FLAGS Display

- ✓ Carry Flag (CF)
- ✓ Zero Flag (ZF)
- ✓ Sign Flag (SF)
- ✓ All status flags

Memory View

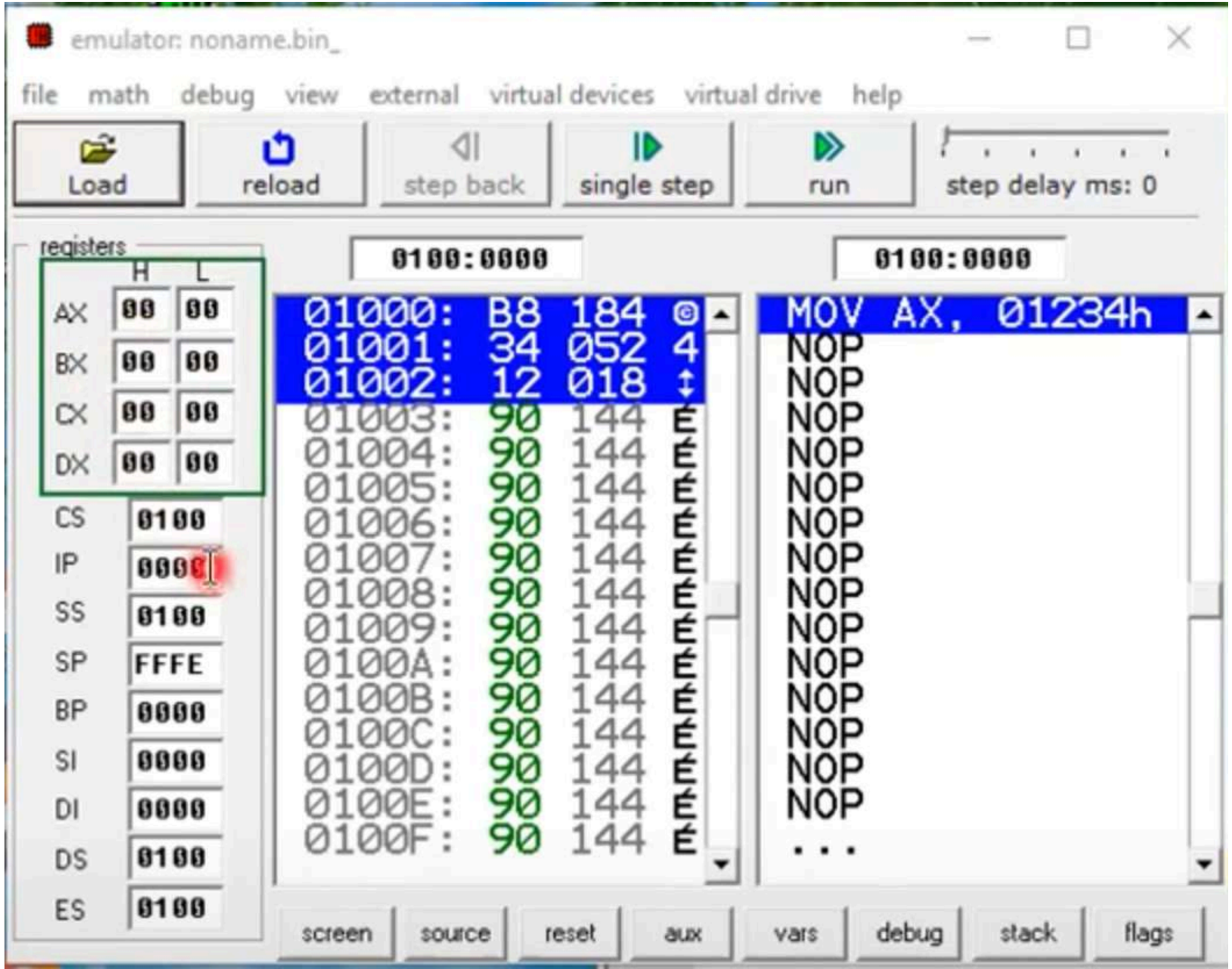
- ✓ Hex dump display
- ✓ ASCII representation
- ✓ Stack visualization
- ✓ Address navigation

Virtual Screen

- ✓ Text output display
- ✓ Character graphics
- ✓ Color support
- ✓ Cursor control

Debugging Tools

- ✓ Step-by-step execution
- ✓ Breakpoints
- ✓ Run to cursor
- ✓ Instruction tracing


emulator: noname.bin_
file math debug view external virtual devices virtual drive help
Load
reload
step back
single step
run
step delay ms: 0
registers
H L
AX 00 00
BX 00 00
CX 00 00
DX 00 00
CS 0100
IP 0000
SS 0100
SP FFFE
BP 0000
SI 0000
DI 0000
DS 0100
ES 0100
0100:0000
01000: B8 184 ©
01001: 34 052 4
01002: 12 018 ↕
01003: 90 144 É
01004: 90 144 É
01005: 90 144 É
01006: 90 144 É
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
0100B: 90 144 É
0100C: 90 144 É
0100D: 90 144 É
0100E: 90 144 É
0100F: 90 144 É
0100:0000
MOV AX, 01234h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
screen source reset aux vars debug stack flags

MOV Instruction Deep Dive

The **MOV** instruction is one of the most fundamental operations in assembly. It copies data from a source to a destination.

Example: MOV AX, 1234H

MOV
Copy value from source to destination

AX
16-bit accumulator register (destination)

1234H
Immediate hexadecimal value (source)

Operation Breakdown

Instruction:
MOV AX, 1234H

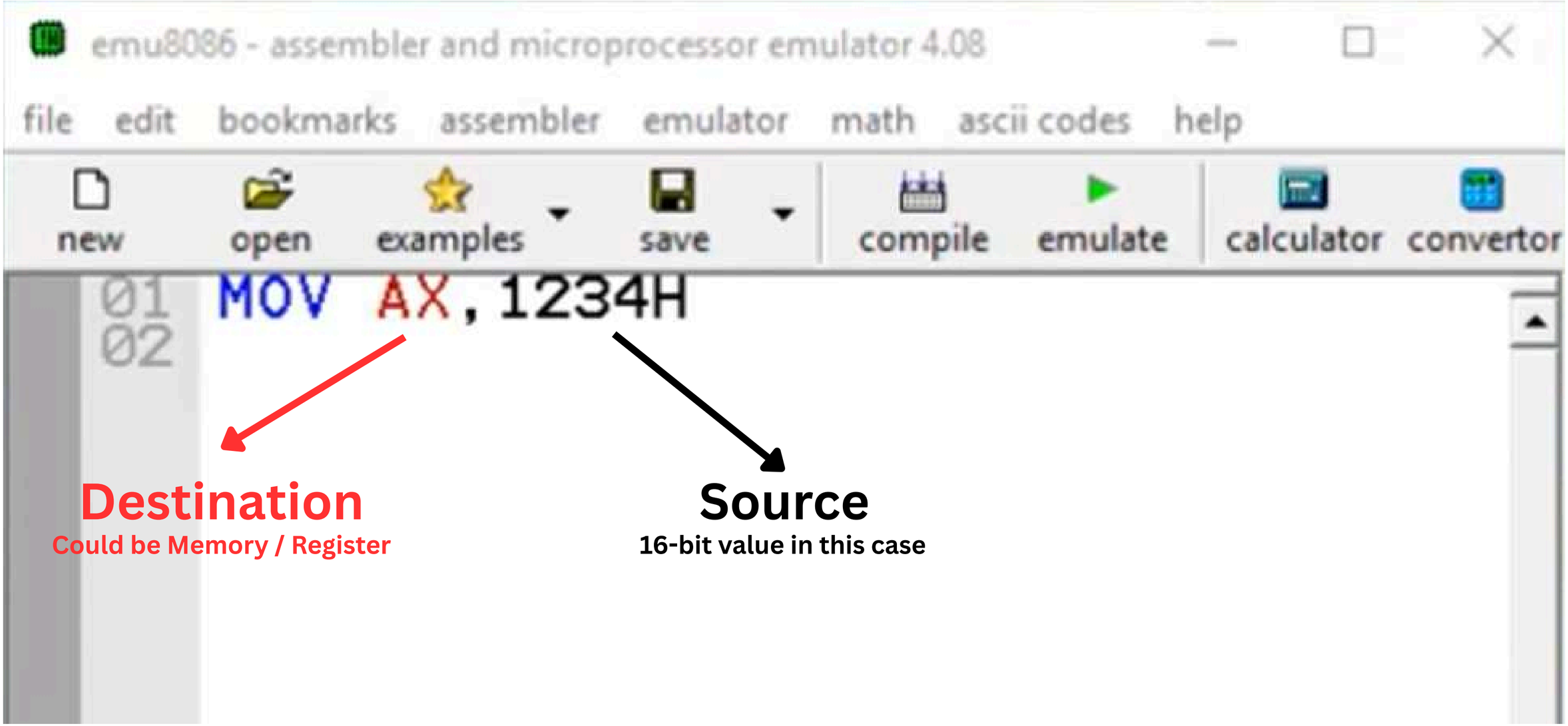
Operation:
Load value 1234H into AX register

Result:
AX = 1234H
AH = 12H (high byte)
AL = 34H (low byte)

Note:
Old contents of AX are overwritten

Important: MOV does NOT affect any flags (ZF, SF, CF, OF remain unchanged).

emu8086


emu8086 - assembler and microprocessor emulator 4.08
file
edit
bookmarks
assembler
emulator
math
ascii codes
help
new
open
examples
save
compile
emulate
calculator
convertor
01 MOV AX,1234H
02
Destination
Could be Memory / Register
Source
16-bit value in this case

MOV Operation Details

After executing **MOV AX, 1234H**, the AX register and its sub-registers contain specific values that can be observed in emu8086.

Register State After MOV

16-bit Register
AX
1234H

High Byte
AH
12H

Low Byte
AL
34H

Relationship
AX = AH:AL
1234 = 12:34

Binary Breakdown

AX (16-bit):
0001 0010 0011 0100

AH (8-bit):
0001 0010

AL (8-bit):
0011 0100

Decimal Value:
1234H = 4660

In emu8086: You can step through to confirm AH = 12H and AL = 34H in the register window.

MOV and Flags

One important characteristic of the **MOV** instruction is that it **does not affect any flags**. This is crucial to understand when writing assembly programs.

Flags Unchanged by MOV

CF Carry Flag	**ZF** Zero Flag
SF Sign Flag	**OF** Overflow Flag
PF Parity Flag	**AF** Auxiliary Flag

Practical Example

Before MOV:

ZF = 1, SF = 0, CF = 0

Execute:

MOV AX, 0

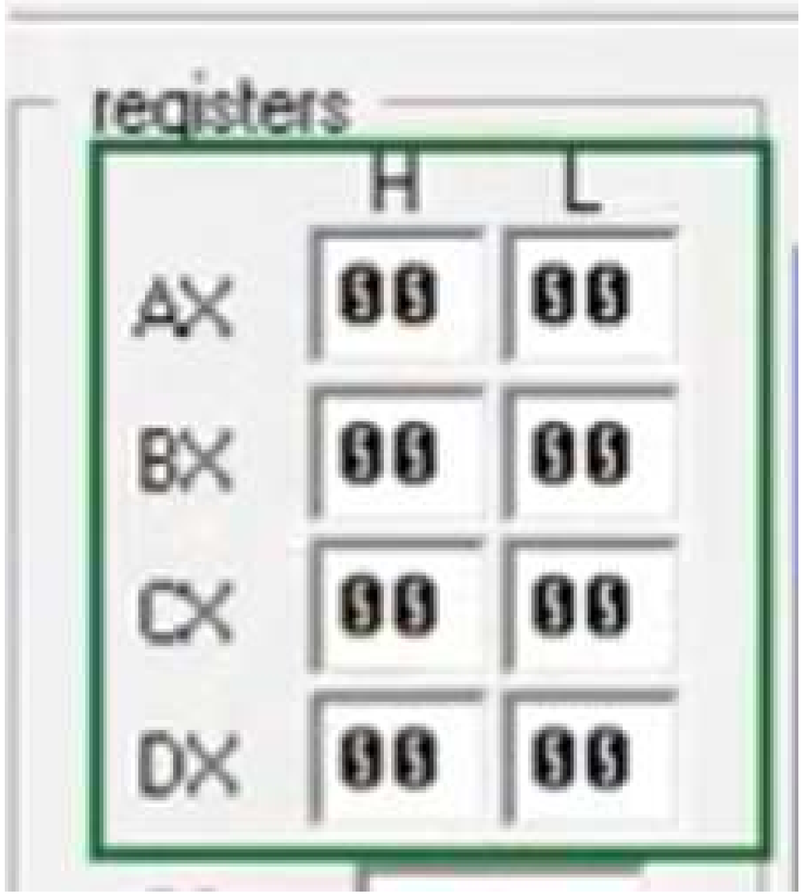
After MOV:

ZF = 1, SF = 0, CF = 0

Flags remain unchanged!

Key Insight: MOV simply transfers data without performing any arithmetic or logical operation, so no flags need to be updated. This is different from ADD, SUB, or other arithmetic instructions.

emu8086



- **AX Register (Accumulator Register):**
  - **H:** High byte of AX (AH register)
  - **L:** Low byte of AX (AL register)
  - Currently contains some value (not shown as 00)
- **Other Registers:**
  - **BX (Base Register):** 00 00 (both high and low bytes are zero)
  - **CX (Count Register):** 00 00 (both high and low bytes are zero)
  - **DX (Data Register):** 00 00 (both high and low bytes are zero)

original source code

```
01 MOV AX,1234H
02 MOV DH,24H
03
04
```

registers

	H	L
AX	12	34
BX	00	00
CX	00	00
DX	24	00
CS	0100	
IP	0005	
SS	0100	
SP	FFFE	

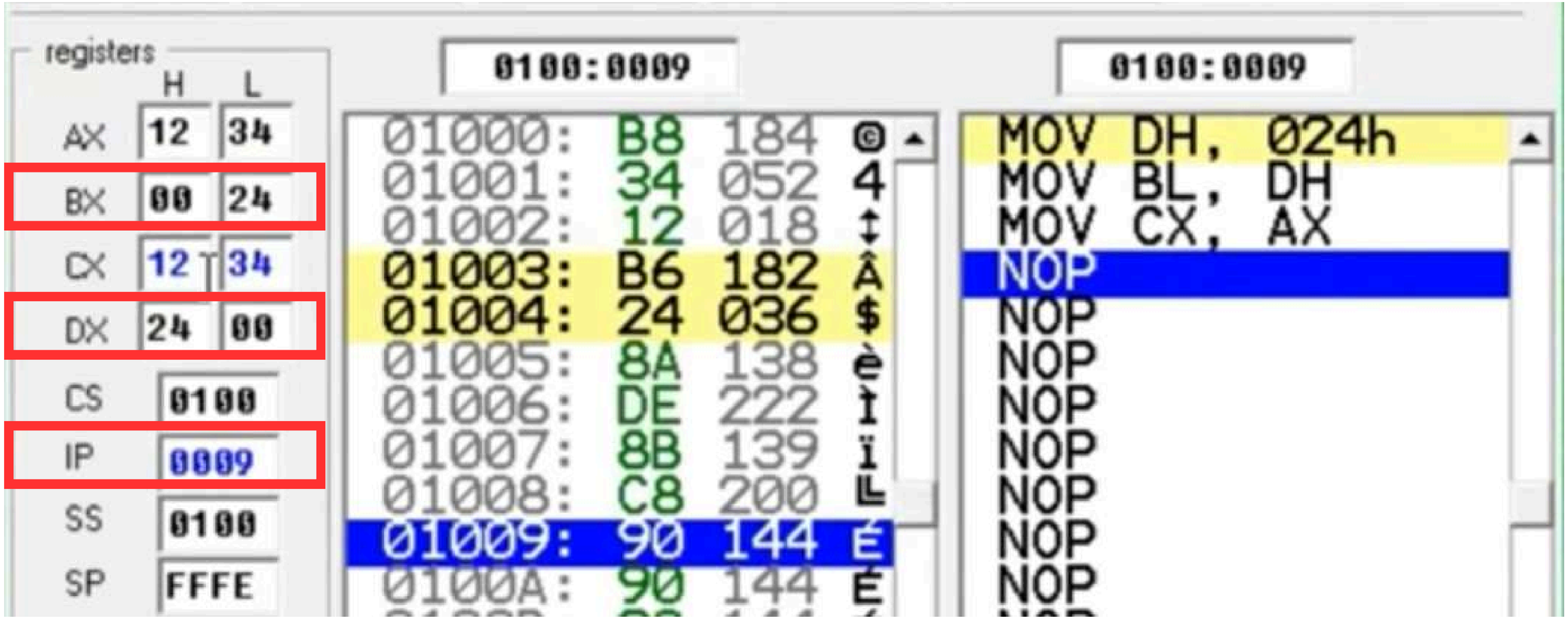
0100:0005

```
01000: B8 184 ©
01001: 34 052 4
01002: 12 018 ↕
01003: B6 182 Â
01004: 24 036 $
01005: 90 144 É
01006: 90 144 É
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
```

0100:0005

```
MOV AX, 01234h
MOV DH, 024h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
```

```
01  MOV AX,1234H
02  MOV DH,24H
03  MOV BL,DH
04  MOV CX,AX
```



04

Understanding Registers

THE WORKING MEMORY OF THE PROCESSOR

What are Registers?

Registers are small, fast storage locations inside the CPU. They serve as the processor's working memory for holding data, addresses, and status information during program execution.

Why Registers Matter

Extremely Fast
Much faster than RAM access (nanoseconds vs. milliseconds)

Inside the CPU
Directly accessible by the processor

Working Storage
Temporary holding area for calculations

Register Types in 8086

General Purpose Used for arithmetic and data operations	AX, BX, CX, DX
Segment Define memory segments	CS, DS, SS, ES
Pointer Track execution and stack	IP, SP, BP
Index Used for string operations	SI, DI

8086 Registers: All registers are 16-bit wide, but can be split into 8-bit halves (high and low bytes).

General Purpose Registers

The 8086 has **four 16-bit general purpose registers**: AX, BX, CX, and DX. Each can be split into high and low 8-bit portions.

AX Accumulator

Primary arithmetic register

AX = AH:AL

- ✓ Default for arithmetic operations
- ✓ Used in multiplication/division
- ✓ I/O operations

BX Base Register

Memory addressing

BX = BH:BL

- ✓ Base address for memory access
- ✓ Array indexing
- ✓ General calculations

CX Count Register

Loop and string operations

CX = CH:CL

- ✓ Loop counter (LOOP instruction)
- ✓ String operation counter
- ✓ Shift/rotate count

DX Data Register

I/O and multiplication

DX = DH:DL

- ✓ I/O port addressing
- ✓ High word in multiplication
- ✓ General data storage

AX Register - The Accumulator

The **AX register** is the primary accumulator for arithmetic operations. It's the default register for many instructions and plays a central role in calculations.

AX Structure

AH (8-bit)		AL (8-bit)		AX (16-bit)
12H	+	34H	=	1234H

- **AH:** High byte (bits 8-15)
- **AL:** Low byte (bits 0-7)

Common Uses of AX

+ Arithmetic Operations

Default register for ADD, SUB, INC, DEC

× Multiplication

MUL result stored in DX:AX

÷ Division

DIV uses AX as dividend

I/O Operations

IN/OUT instructions use AL/AX

Tip: Many instructions have optimized forms when using AX, making your code smaller and faster!

BX, CX and DX Registers

BX Base

Memory addressing and array indexing

Structure:
BX = BH:BL

- Base address calculations
- Array element access
- Pointer operations

```
mov ax, [bx]
```

CX Count

Loop counter and string operations

Structure:
CX = CH:CL

- LOOP instruction counter
- String repeat count
- Shift/rotate iterations

```
loop label
```

DX Data

I/O operations and extended arithmetic

Structure:
DX = DH:DL

- I/O port addressing
- High word in MUL
- Remainder in DIV

```
in ax, dx
```

Remember: While these registers have specialized purposes, they can all be used for general arithmetic when needed. The specialized uses are just optimizations!

Segment Registers Overview

Segment registers define memory segments in the 8086 processor. They work with offset registers to form complete memory addresses, enabling access to up to 1MB of memory.

CS Code Segment

Points to the memory segment where program code is stored.

Combined with IP:
CS:IP = next instruction

Example: CS = 0100, IP = 0000 → Address 0100:0000

DS Data Segment

Points to the memory segment for data operations.

Default for data access:
mov ax, [variable]

Example: DS = 0100 → Data at 0100:0000

SS Stack Segment

Points to the memory segment for the stack.

Combined with SP:
SS:SP = top of stack

Example: SS = 0100, SP = FFFE → Stack top

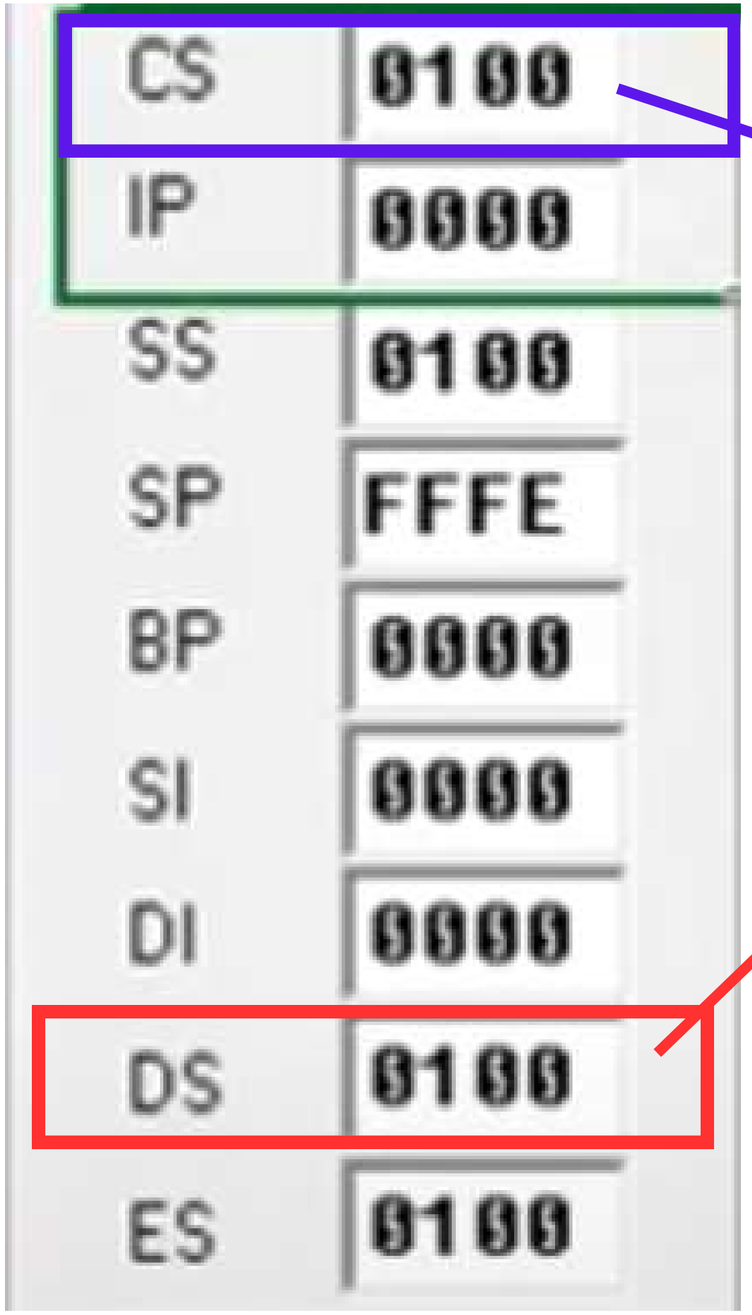
ES Extra Segment

Additional data segment for string operations.

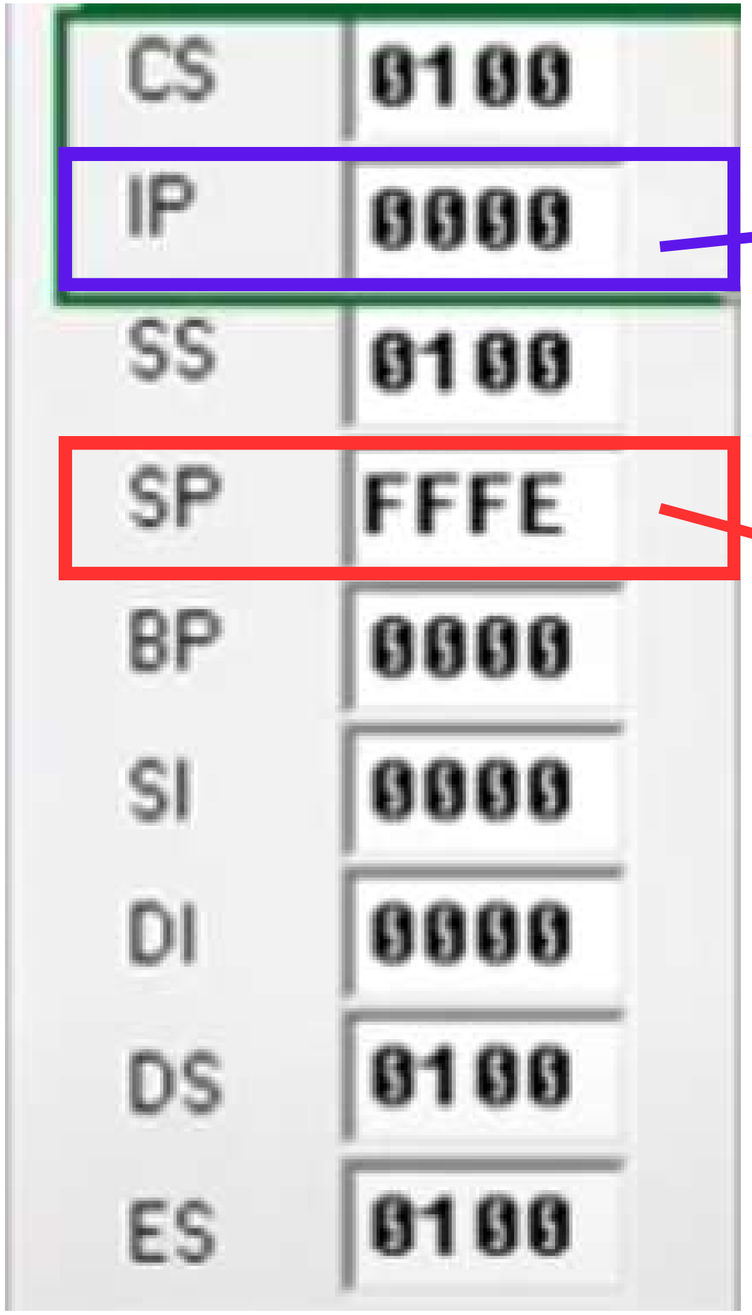
Used with string ops:
movsb (DS:SI to ES:DI)

Example: ES = 0200 → Destination for strings

Segment Registers



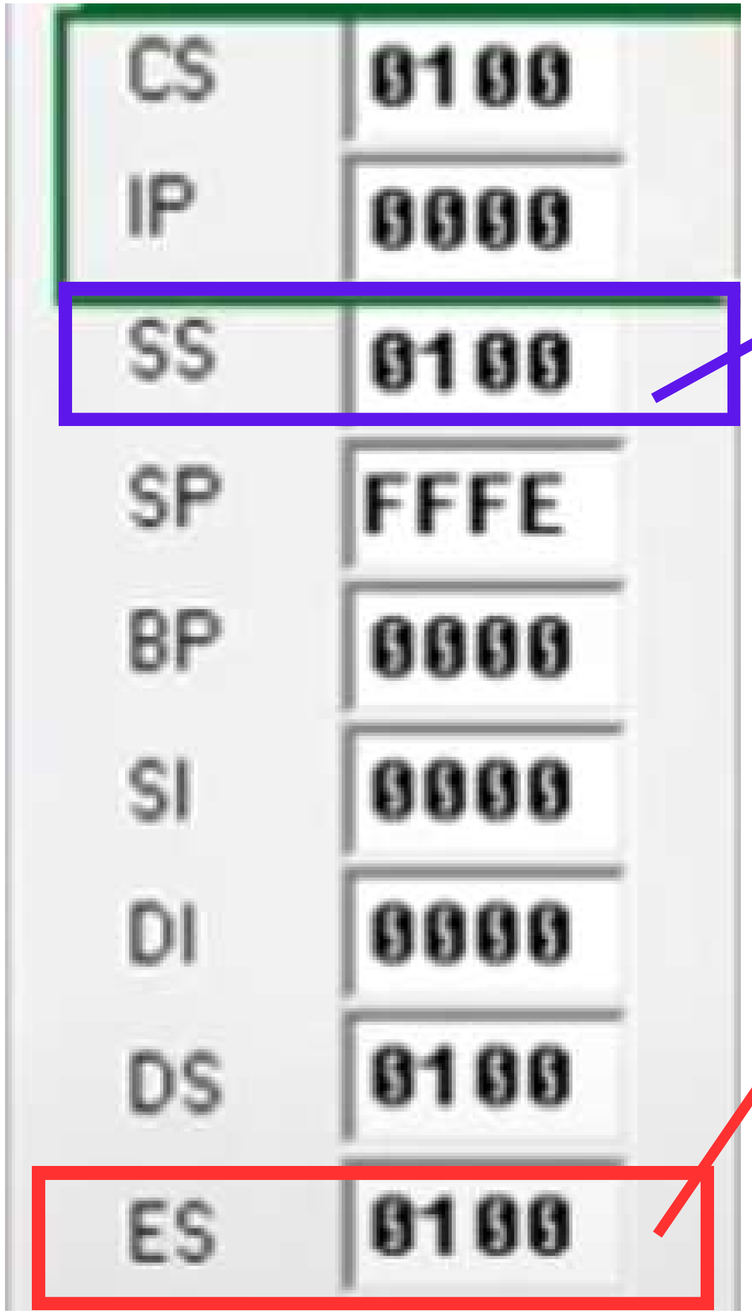
- **CS (Code Segment): 0100**
  - Points to the memory segment where the program code is stored
  - Current code segment starts at address 0100:0000 (shifted: 01000h)
- **DS (Data Segment): 0100**
  - Points to the memory segment for data operations
  - Data segment starts at address 0100:0000



Pointer / Index Registers

- **IP (Instruction Pointer): 0000**
  - Points to the next instruction to execute within the code segment
  - Combined with CS: CS:IP = 0100:0000
- **SP (Stack Pointer): FFFE**
  - Points to the top of the stack within the stack segment
  - Combined with SS: SS:SP = 0100:FFFE
  - Indicates the stack grows downward from the top of the segment

Segment Registers



- **SS (Stack Segment): 0100**
  - Points to the memory segment for the stack
  - Stack segment starts at address 0100:0000
- **ES (Extra Segment): 0100**
  - Additional data segment used for string operations
  - Also starts at 0100:0000

Pointer Registers

IP Instruction Pointer

Purpose

Points to the next instruction to execute within the code segment. Automatically incremented after each instruction.

Address Format:

CS:IP = next instruction

Example: 0100:0000

Initial Value: 0000H (start of program)

SP Stack Pointer

Purpose

Points to the top of the stack within the stack segment. Changes with PUSH and POP operations.

Stack Operations:

PUSH: SP = SP - 2
POP: SP = SP + 2

Initial Value: FFFE (top of segment)

How They Work Together

Instruction Execution

1. CPU reads CS:IP
2. Fetches instruction
3. Executes instruction
4. IP increments automatically

Stack Management

1. SS:SP = stack top
2. PUSH: data stored, SP ↓
3. POP: data retrieved, SP ↑
4. Stack grows downward

05

Assembly Instructions

THE COMMANDS THAT DRIVE THE PROCESSOR

ADD Instruction Overview

The **ADD** instruction performs arithmetic addition. It adds the source operand to the destination operand and stores the result in the destination.

Basic Syntax

```
ADD destination, source
```

Operation: destination = destination + source

Result: Stored in destination

Source: Unchanged

Valid Operand Combinations

Register + Register

```
add ax, bx
```

Register + Immediate

```
add ax, 5
```

Memory + Register

```
add [var], ax
```

Register + Memory

```
add ax, [var]
```

Invalid: Cannot add memory to memory directly!

ADD Example: Register to Register

Let's walk through a complete example of adding two registers together using the **ADD** instruction.

Example Code

```
; Initialize registers
MOV AX, 1234H
MOV BX, 5678H

; Perform addition
ADD BX, AX
```

Step-by-Step Execution

Step 1: After MOV AX, 1234H

AX = 1234H
BX = 0000H

Step 2: After MOV BX, 5678H

AX = 1234H
BX = 5678H

Step 3: After ADD BX, AX

BX = 5678H + 1234H

BX = 68ACH

Note: AX remains unchanged (1234H), only BX is updated with the sum.

Binary Calculation

BX	AX	Result
0101 0110 0111 1000	0001 0010 0011 0100	0110 1000 1010 1100
5678H	1234H	**68ACH**

ADD with Immediate Values

You can also add **immediate values** (constants) directly to registers. This is useful for adding fixed amounts without loading them into a register first.

Example: ADD BH, 24H

```
; Initial state (after previous ADD)
BX = 68ACH
BH = 68H

; Add immediate value
ADD BH, 24H
```

Execution Details

Before:
BH = 68H (0110 1000)

Operation:
68H + 24H = 8CH

After:
BH = 8CH (1000 1100)
BL = ACH (unchanged)

Key Point: Only BH is affected. BL remains unchanged (ACH), and the full BX becomes 8ACH.

Binary Addition

BH	+	Immediate	=	Result
0110 1000		0010 0100		1000 1100
68H		24H		**8CH**

SUB Instruction Overview

The **SUB** instruction performs arithmetic subtraction. It subtracts the source operand from the destination operand and stores the result in the destination.

Basic Syntax

```
SUB destination, source
```

Operation: destination = destination - source

Result: Stored in destination

Source: Unchanged

Valid Operand Combinations

Register - Register

```
sub ax, bx
```

Register - Immediate

```
sub ax, 5
```

Memory - Register

```
sub [var], ax
```

Register - Memory

```
sub ax, [var]
```

Two's Complement: SUB is implemented as addition of the two's complement.

SUB Example: Basic Subtraction

Let's examine a basic subtraction example to understand how the **SUB** instruction works with 8-bit registers.

Example Code

```
; Initialize registers
MOV AH, 0CH
MOV BL, 08H

; Perform subtraction
SUB AH, BL
```

Step-by-Step Execution

Step 1: After MOV AH, 0CH

AH = 0CH (12 decimal)

Step 2: After MOV BL, 08H

BL = 08H (8 decimal)

Step 3: After SUB AH, BL

AH = 0CH - 08H

AH = 04H

(4 decimal)

Note: BL remains unchanged (08H), only AH is updated with the difference.

Binary Subtraction

AH	–	BL	=	Result
0000 1100 0CH		0000 1000 08H		0000 0100 **04H**

SUB with Immediate Values

Subtracting immediate values from 16-bit registers works similarly. Let's examine a more complex example.

Example: SUB CX, 456AH

```
; Initialize CX
MOV CX, 4578H

; Subtract immediate
SUB CX, 456AH
```

Execution Details

Before:
CX = 4578H

Operation:
4578H - 456AH

After:
CX = 000EH
(14 decimal)

Calculation: 4578H (17784) - 456AH (17770) = 000EH (14)

Binary Subtraction with Borrow

Low Byte Subtraction:

78H - 6AH
0111 1000
- 0110 1010
= 0000 1110 (0EH)

Borrow needed from high byte!

High Byte Subtraction:

45H - 45H
0100 0101
- 0100 0101
= 0000 0000 (00H)

With borrow: 44H - 45H

Two's Complement Subtraction

In digital systems, **subtraction is performed as addition of the two's complement**. This simplifies the hardware design.

Two's Complement Steps

Step 1: Invert all bits

Change 0s to 1s and 1s to 0s (one's complement)

Step 2: Add 1

Add 1 to the inverted value

Example: 5 - 3

Original:

5 = 0000 0101
3 = 0000 0011

Two's complement of 3:

Invert: 1111 1100
Add 1: 1111 1101 (-3)

Addition:

```
  0000 0101 (5)
+ 1111 1101 (-3)
= 0000 0010 (2)
```

Key Insight: The CPU uses the same ADD circuitry for both addition and subtraction!

INC and DEC Instructions

INC Increment

Operation

Adds 1 to the operand

operand = operand + 1

Examples:

```
inc ax ; AX = AX + 1
inc [var] ; Memory++
```

Flag Effects: Affects all except CF

DEC Decrement

Operation

Subtracts 1 from the operand

operand = operand - 1

Examples:

```
dec bx ; BX = BX - 1
dec cx ; CX = CX - 1
```

Flag Effects: Affects all except CF

Common Use Case: Loop Counter

Using DEC:

```
mov cx, 10
loop_start:
    ; ... do something
    dec cx
    jnz loop_start
```

Using LOOP:

```
mov cx, 10
loop_start:
    ; ... do something
    loop loop_start
```

LOOP automatically decrements CX!

06

FLAGS Register Deep Dive

CHAPTER 1: Processor State

UNDERSTANDING THE PROCESSOR STATUS RE

Carry Flag (CF)

CF (Carry Flag) indicates a carry or borrow in unsigned arithmetic operations.

When CF is Set

Addition

CF = 1 if result exceeds maximum value (carry out of MSB)

FFH + 1 = 00H, CF=1

Subtraction

CF = 1 if borrow needed (result < 0 for unsigned)

5 - 10, CF=1

Examples

Example 1 - Addition:

ADD AL, 80H
80H + 80H = 00H
CF = 1 (carry out)

Example 2 - Subtraction:

SUB AL, 5
(AL=3) 3 - 5
CF = 1 (borrow)

Usage: Detect overflow in unsigned arithmetic, multi-precision operations.

Zero Flag (ZF)

ZF (Zero Flag) is set when the result of an operation is zero. It's one of the most frequently used flags for conditional operations.

ZF Behavior

ZF = 1

Result is zero

5 - 5 = 0, ZF=1

ZF = 0

Result is not zero

5 + 3 = 8, ZF=0

Common Use Cases

Equality Check

CMP AX, BX followed by JE (jump if equal)

Loop Termination

DEC CX followed by JZ (jump if zero)

Search Operations

Check if value found (zero indicates match)

Example: CMP AX, 5 → JE label (jump if AX equals 5)

Sign Flag (SF)

SF (Sign Flag) indicates the sign of the result for signed arithmetic. It's set to the value of the most significant bit (MSB) of the result.

SF Behavior

SF = 1

Result is negative (MSB = 1)

8CH = 1000 1100, SF=1

SF = 0

Result is positive (MSB = 0)

68H = 0110 1000, SF=0

8-bit vs 16-bit

8-bit operation:

SF = bit 7

Range: -128 to +127

16-bit operation:

SF = bit 15

Range: -32768 to +32767

Important: SF only meaningful for signed arithmetic interpretation!

Visual Representation

Positive Number

0xxx xxxx

SF = 0

VS

Negative Number

1xxx xxxx

SF = 1

Overflow Flag (OF)

OF (Overflow Flag) indicates overflow in signed arithmetic. It detects when the result exceeds the range of signed numbers.

When OF is Set

Positive + Positive = Negative

Two positives overflow into negative

Negative + Negative = Positive

Two negatives overflow into positive

Example: 8-bit Overflow

Signed range: -128 to +127

Operation:
68H (104) + 24H (36)
= 8CH (-116)

OF = 1 (overflow!)
104 + 36 = 140 > 127

⚠ **Critical:** OF for signed, CF for unsigned!

OF vs CF Comparison

Overflow Flag (OF)

- For signed arithmetic
- Detects sign change overflow
- Range exceeded (signed)

Carry Flag (CF)

- For unsigned arithmetic
- Detects carry/borrow
- Range exceeded (unsigned)

Parity Flag (PF)

PF (Parity Flag) indicates even parity of the least significant byte of the result. It's set if the number of **1-bits** is even.

PF Behavior

PF = 1

Even number of 1-bits in low byte

PF = 0

Odd number of 1-bits in low byte

Examples

Example 1:

AL = 03H = 0000 0011

Two 1-bits → PF = 1

Example 2:

AL = 8CH = 1000 1100

Three 1-bits → PF = 0

Example 3:

AL = 00H = 0000 0000

Zero 1-bits → PF = 1

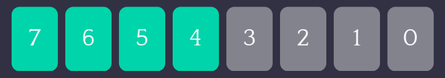
Usage: Error detection in data transmission (parity checking).

Auxiliary Carry Flag (AF)

AF (Auxiliary Carry Flag) indicates a carry or borrow between the **low nibble** (bits 0-3) and **high nibble** (bits 4-7). Used primarily for BCD arithmetic.

Nibble Visualization

8-bit Byte Structure



High Nibble | Low Nibble

AF detects carry between nibbles

$$
\begin{array}{r}
1\,1\,1 \\
0\,0\,1\,1\;1\,0\,0\,0 \\
+\,0\,0\,1\,1\;1\,1\,0\,1 \\
\hline
0\,1\,1\,1\;0\,1\,0\,1
\end{array}
$$

Here, the result is $(0111\ 0101)_2 = (75)_H$

In this addition, observe the carry generated from the addition of 4 LSBs (1000 + 1101) and it results in a carry forward to the next 4 MSBs. Since, there is a carry generated between the lower and upper nibbles, thus it sets the AF to 1.

Interrupt Enable Flag (IF)

IF (Interrupt Enable Flag) controls whether the processor responds to maskable interrupts. It's a control flag that affects system behavior.

IF States

IF = 1 (Set)

Interrupts are enabled
CPU responds to interrupt requests

IF = 0 (Cleared)

Interrupts are disabled
CPU ignores maskable interrupts

Common Use Cases

Critical Code Sections

Disable interrupts during time-sensitive operations

Atomic Operations

Prevent interruption during multi-step operations

Interrupt Handlers

Control when interrupts can occur

Instructions:

STI → Set IF (enable)
CLI → Clear IF (disable)

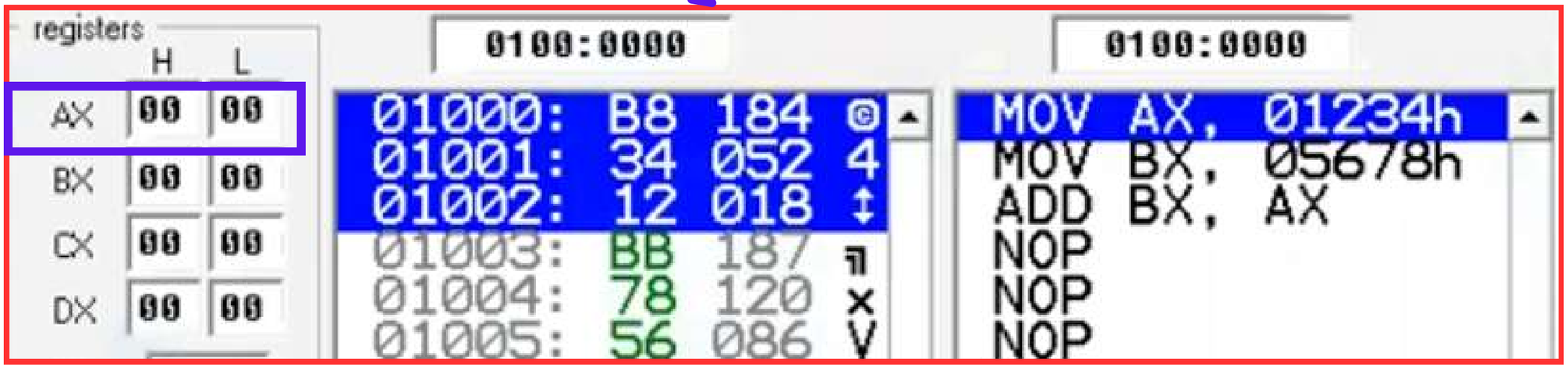
Flag Summary Table

Flag	Name	Purpose	Set When
CF	Carry Flag	Unsigned overflow	Carry/borrow out of MSB
PF	Parity Flag	Even parity check	Even number of 1s in low byte
AF	Auxiliary Carry	BCD arithmetic	Carry/borrow between nibbles
ZF	Zero Flag	Zero result	Result equals zero
SF	Sign Flag	Signed result sign	MSB of result is 1
OF	Overflow Flag	Signed overflow	Signed result overflows
IF	Interrupt Enable	Interrupt control	Software controlled

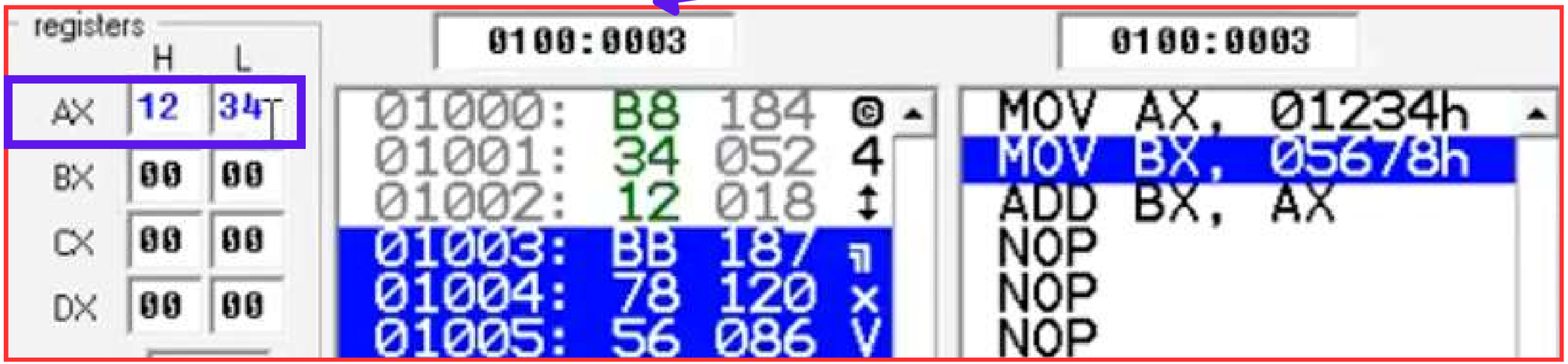
1

```
MOV AX, 1234H
MOV BX, 5678H
ADD BX, AX
```

2

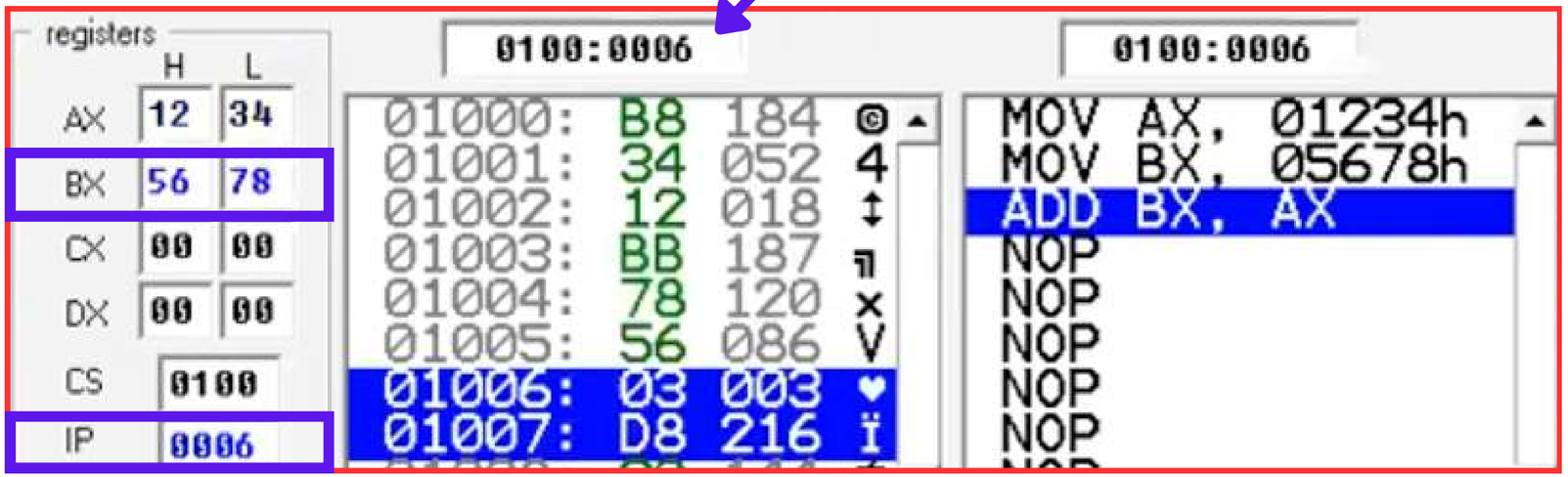


3

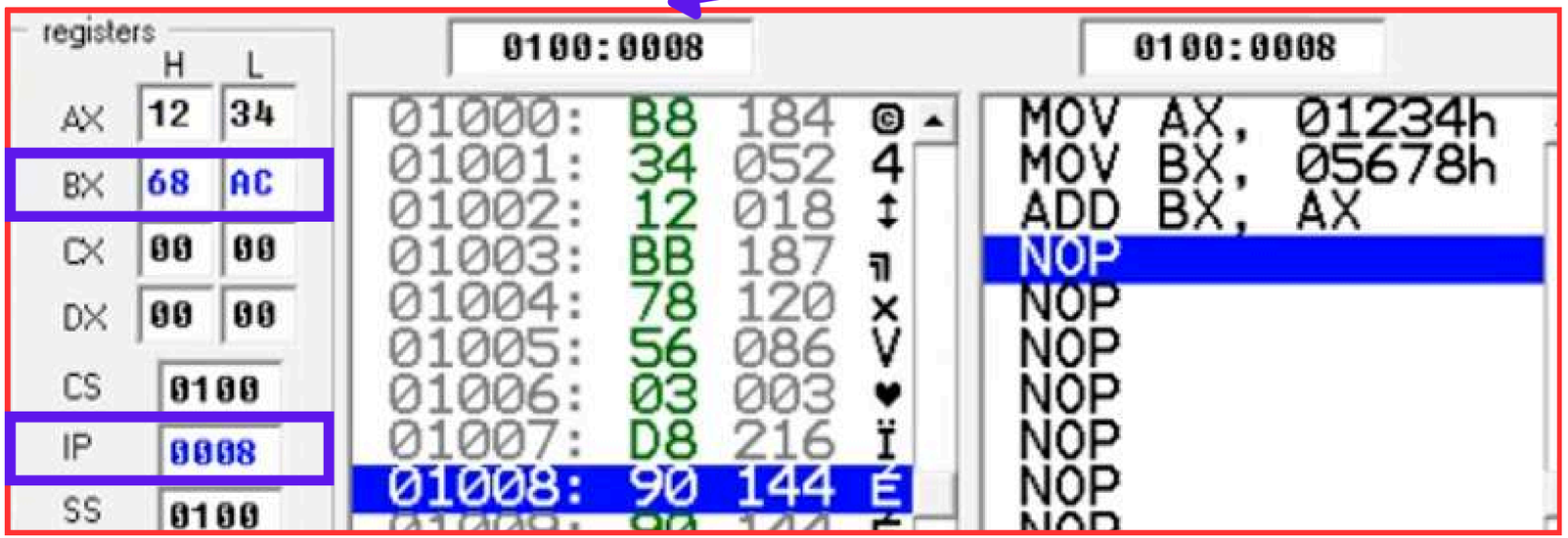


```
MOV AX, 1234H
MOV BX, 5678H
ADD BX, AX
```

4

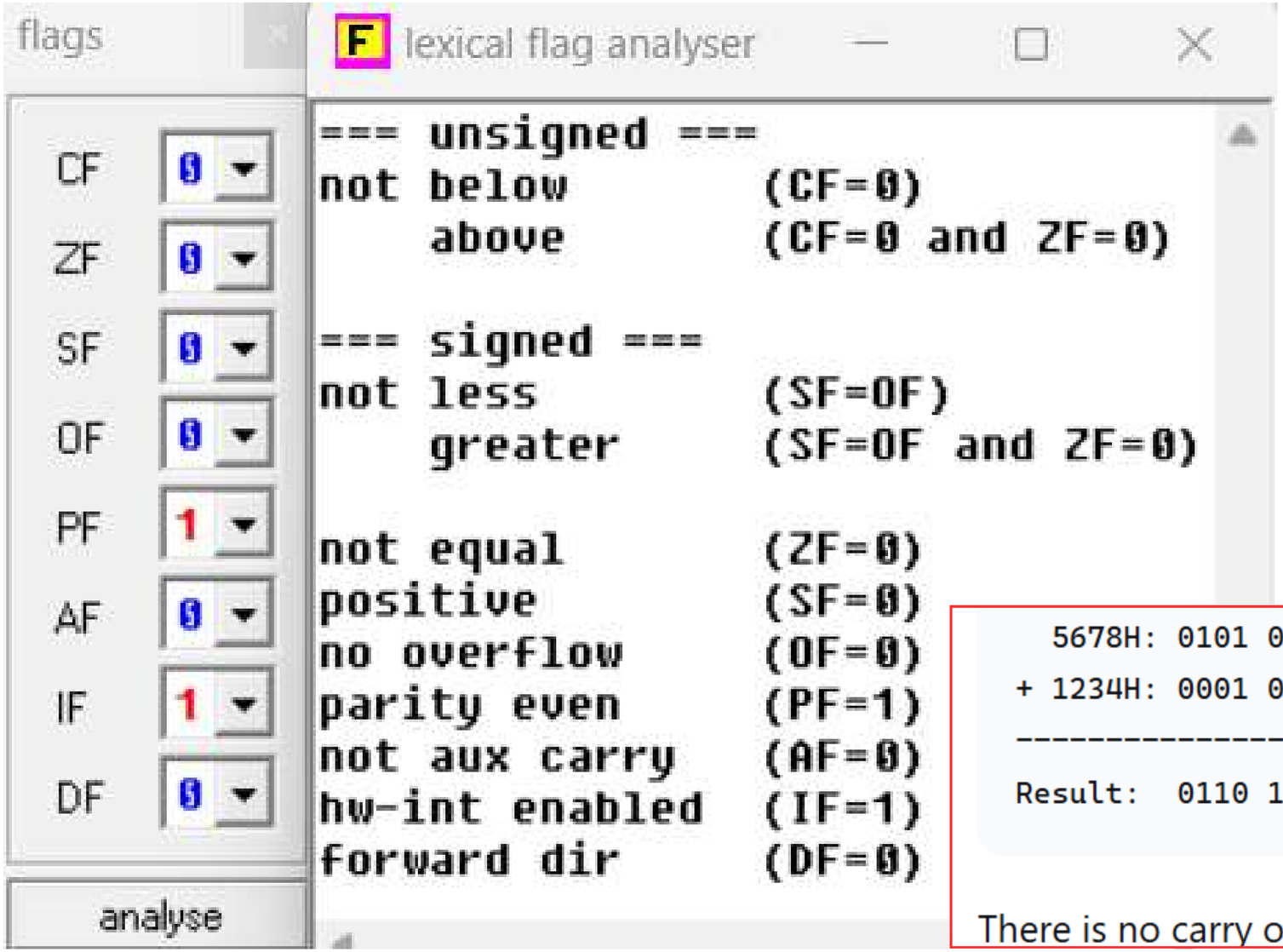


ADD



Flags Affected ADD BX, AX

```
MOV AX, 1234H
MOV BX, 5678H
ADD BX, AX
```

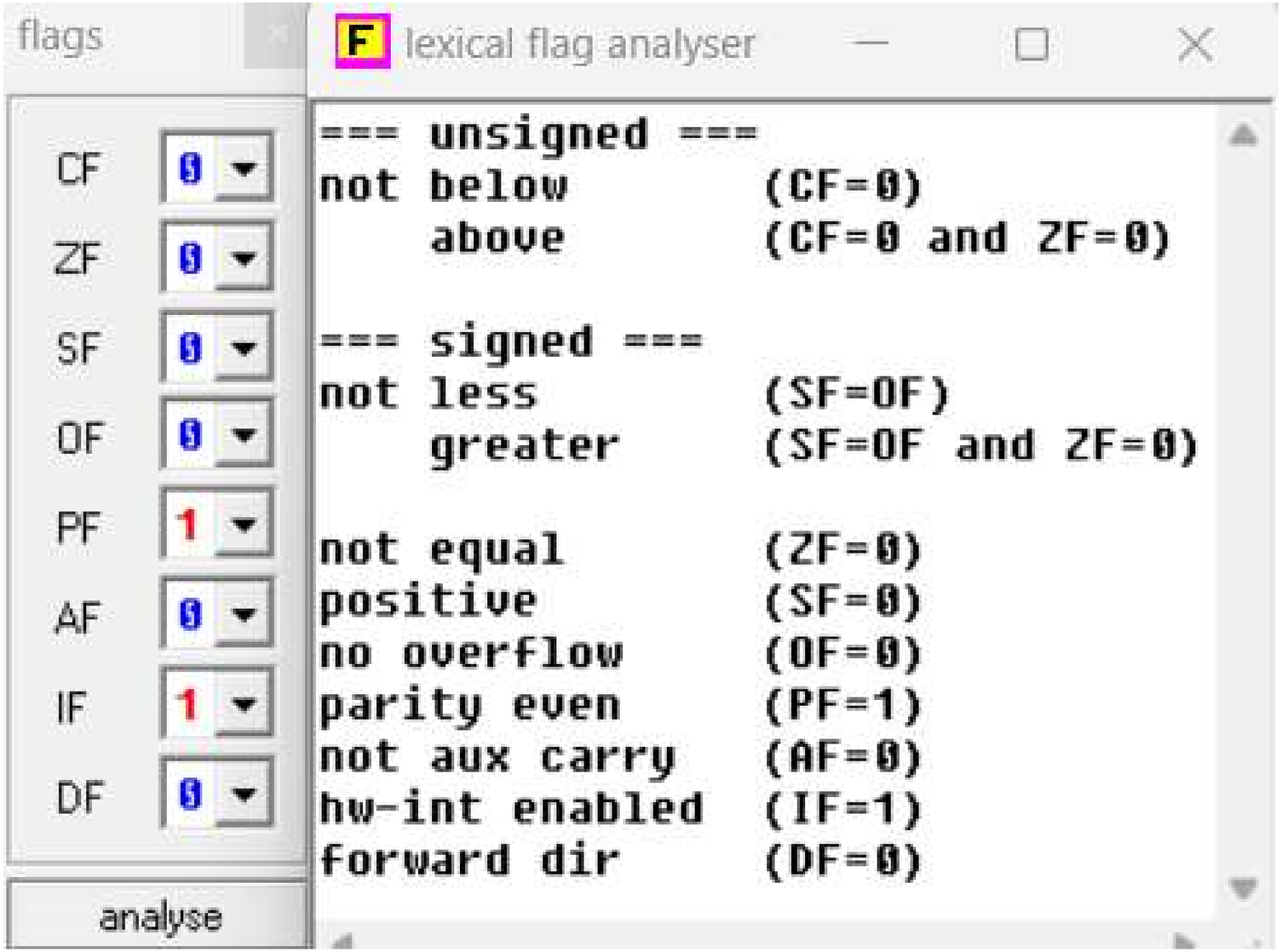


The **Zero Flag (ZF)** will be cleared (**0**) because the result (68ACH) is not zero.

```
  5678H: 0101 0110 0111 1000
+ 1234H: 0001 0010 0011 0100
-----------------------------
Result:  0110 1000 1010 1100  (68ACH)
```

There is no carry out from the most significant bit (bit 15), so CF = 0.

Flags Affected ADD BX, AX

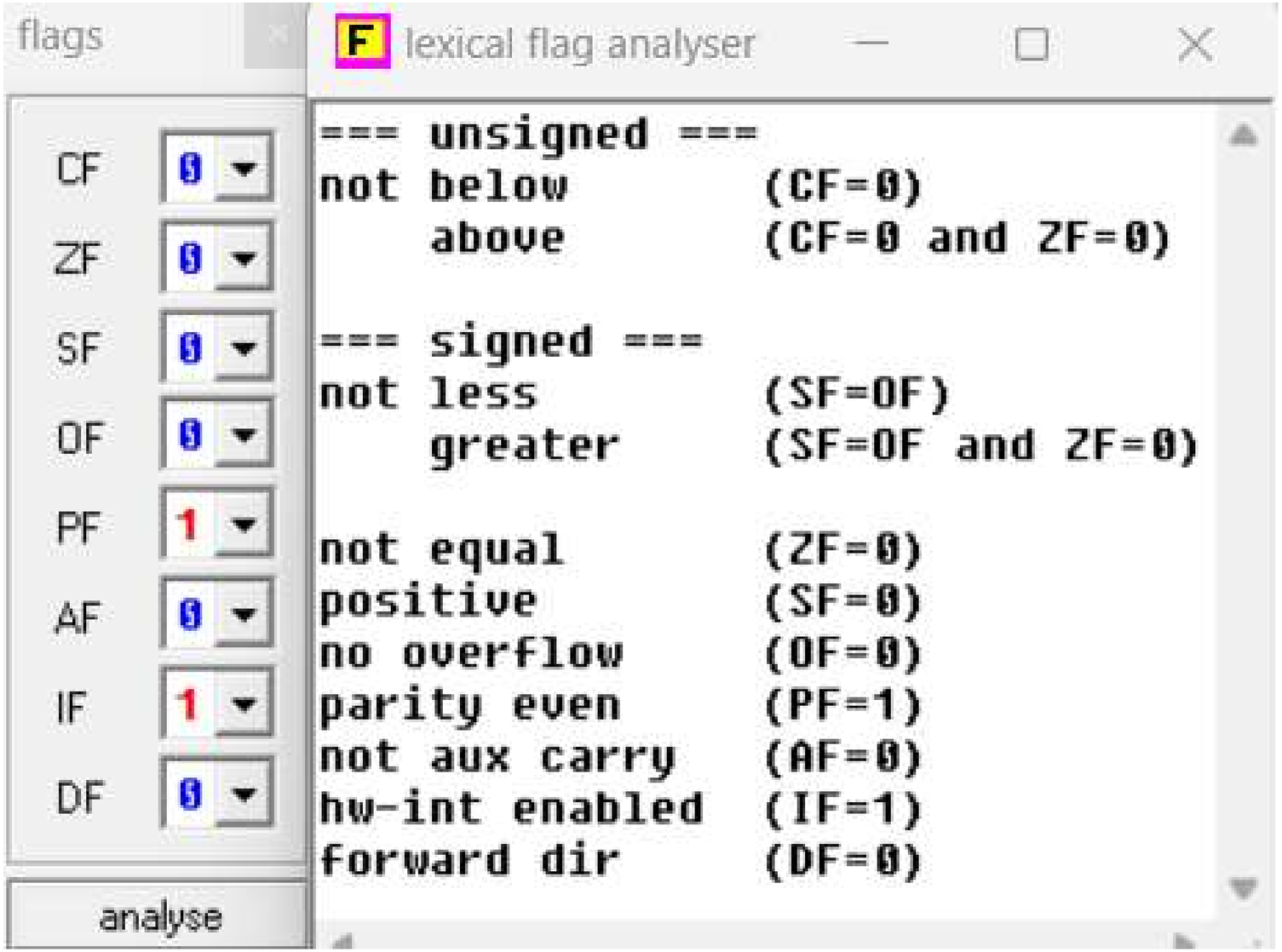



```
MOV AX, 1234H
MOV BX, 5678H
ADD BX, AX
```

Sign Flag (SF): Set (1) if the result is negative (i.e., the most significant bit (MSB) is 1). Cleared (**0**) if positive.

So, 0110 1000 1010 1100 (16 bits).

Flags Affected ADD BX, AX




```
MOV AX, 1234H
MOV BX, 5678H
ADD BX, AX
```

The binary representation of the result (68ACH) is: **So, 0110 1000 1010 1100 (16 bits).**

The **Parity Flag (PF) is set (1)** if the number of 1s in the **least significant byte** (lower 8 bits of 16-bit register) of the result is even.

Step 1: Compute the full result

AF = 0

- `5678H + 1234H = 68ACH`

Step 2: Focus on the low byte addition (78H + 34H)

- Low byte of BX: 78H
- Low byte of AX: 34H
- Add: 78H + 34H = ACH (in low byte of result)

```
Bit: 7654 3210
  78H = 0111 1000
+ 34H = 0011 0100
---------------
  ACH = 1010 1100
```

AF = 0

Now, check for carry from bit 3 to bit 4:

- Bit 3 (rightmost bit of the higher nibble) is the 4th bit (counting from 0).
- In 78H, bit 3 is 1 (from 0111 **1**000)
- In 34H, bit 3 is 0 (from 0011 **0**100)
- Adding bit 3: 1 + 0 = 1, with no carry from lower bits

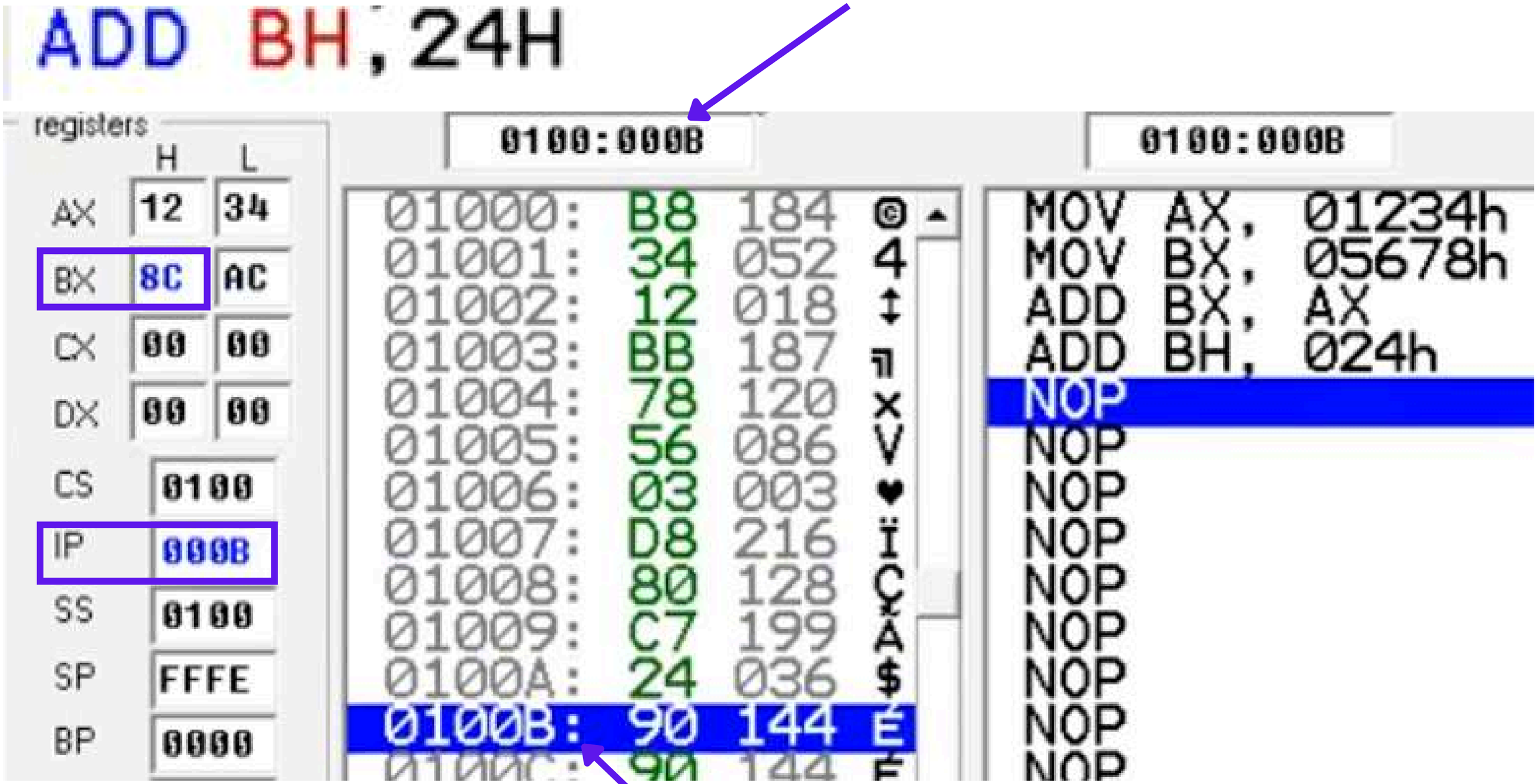
OF = 0

- In signed 16-bit two's complement, the range is from **-32768 to 32767**.
  - 1234H = 4660 (positive)
  - 5678H = 22136 (positive)
  - Sum = 4660 + 22136 = **26796**
  - Since **26796** is within the positive range (0 to 32767), there is no overflow.

```
ADD BH,24H
```

- **BH** is the high byte of the 16-bit BX register (BX = BH:BL).
- 24H is an immediate value (hexadecimal 24).
  - Adds 24H to the current value in BH.
  - Stores the result back in BH.
  - The low byte BL remains unchanged.
  - The flags (e.g., Carry, Zero, Sign) in the FLAGS register are updated based on the result.

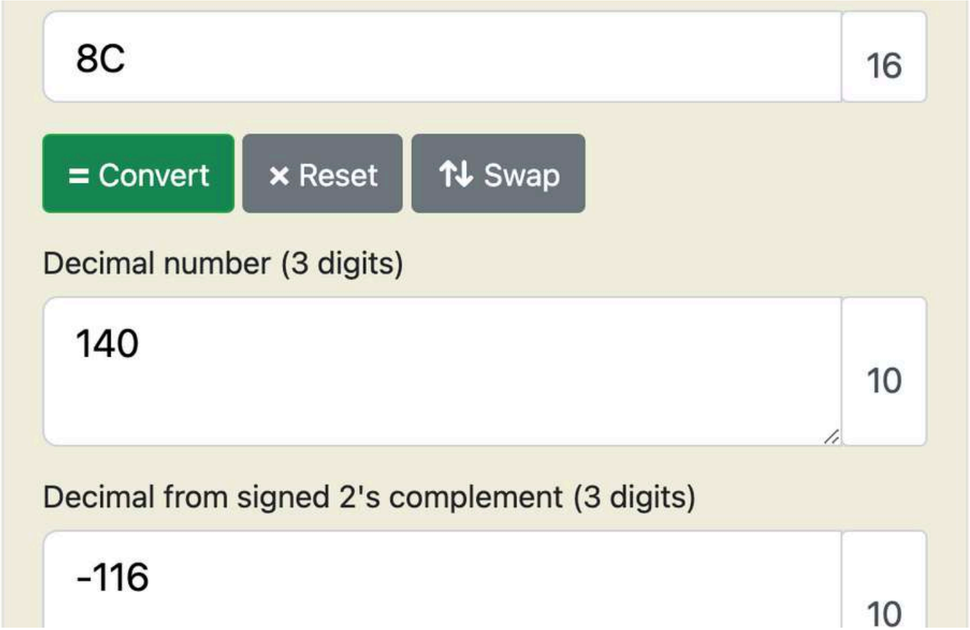
ADD BH, 24H


registers
H L
AX 12 34
BX 8C AC
CX 00 00
DX 00 00
CS 0100
IP 000B
SS 0100
SP FFFE
BP 0000
0100:000B
01000: B8 184
01001: 34 052
01002: 12 018
01003: BB 187
01004: 78 120
01005: 56 086
01006: 03 003
01007: D8 216
01008: 80 128
01009: C7 199
0100A: 24 036
0100B: 90 144
0100:000B
MOV AX, 01234h
MOV BX, 05678h
ADD BX, AX
ADD BH, 024h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP

Flags Affected ADD BH, 24H

- **68H + 24H = 8CH** (no carry, so **CF=0**)
- Result is not zero **(ZF=0)**
- 8CH has MSB = 1 (negative in signed 8-bit), so **SF=1**
- **For signed arithmetic:** 68H (104) + 24H (36) = 8CH (-116) → overflow **(OF=1)** because 104 + 36 = 140, which **exceeds 127** (max positive for 8-bit signed).

```
  0110 1000   (68H)
+ 0010 0100   (24H)
---------------
  1000 1100   (8CH)
```



Flags Affected ADD BH, 24H

- **Auxiliary carry:** 8 + 4 = 12 (hex), so no carry from bit 3 to bit 4 **(AF=0)**
- Since the number of 1s in (8CH) is odd (3), the Parity Flag (**PF**) is cleared (**0**).

Bit: 7654 3210

```
  0110 1000   (68H)
+ 0010 0100   (24H)
---------------
  1000 1100   (8CH)
```

flags

CF	0
ZF	0
SF	1
OF	1
PF	0
AF	0
IF	1
DF	0

analyse

lexical flag analyser

```
=== unsigned ===
not below          (CF=0)
    above          (CF=0 and ZF=0)

=== signed ===
not less           (SF=OF)
    greater        (SF=OF and ZF=0)

not equal          (ZF=0)
negative           (SF=1)
   overflow        (OF=1)
parity odd         (PF=0)
not aux carry      (AF=0)
hw-int enabled     (IF=1)
forward dir        (DF=0)
```

original source c...

```
01 MOV AX, 1234H
02 MOV BX, 5678H
03 ADD BX, AX
04 ADD BH, 24H
05
06
```

```
MOV CX,4578H
SUB CX,456AH
```

SUBTRACT

- **MOV CX, 4578H**
  - Moves the hexadecimal value 4578H into the CX register
  - CX is a 16-bit general-purpose register
  - After this instruction: **CX = 4578H**
- **SUB CX, 456AH**
  - Subtracts the immediate hexadecimal value 456AH from the **CX** register
  - Stores the result back in **CX**
  - Performs: CX = CX - 456AH = 4578H - 456AH

```
MOV CX,4578H
SUB CX,456AH
```

SUBTRACT

```
  4578H
- 456AH
--------
  000EH
```

78H = 120 decimal

6AH = 106 decimal

120 - 106 = 14 decimal = 0EH (no borrow needed for the lower byte)

45H - 45H = 00H (no borrow needed for the upper byte)

So: **4578H - 456AH = 000EH**

flags

CF	0
ZF	0
SF	0
OF	0
PF	0
AF	1
IF	1
DF	0

analyse

lexical flag analyser

```
=== unsigned ===
not below          (CF=0)
    above          (CF=0 and ZF=0)

=== signed ===
not less           (SF=OF)
    greater        (SF=OF and ZF=0)

not equal          (ZF=0)
positive           (SF=0)
no overflow        (OF=0)
parity odd         (PF=0)
    aux carry      (AF=1)
hw-int enabled     (IF=1)
forward dir        (DF=0)
```

original source c...

```
01 MOV CX, 4578H
02 SUB CX, 456AH
03
04
```

- **CF (Carry Flag)** → Set if a **borrow** is needed (i.e., source > destination before subtraction).
  - Here: `4578h > 456Ah`, so **no borrow** → CF = 0.
- **ZF (Zero Flag)** → Set if result = 0.
  - Result = `000Eh` → not zero → ZF = 0.
- **SF (Sign Flag)** → Set if result MSB = 1 (negative in signed interpretation).
  - `000Eh` = `0000 0000 0000 1110b`, MSB = 0 → SF = 0.
- **OF (Overflow Flag)** → Set if signed overflow occurs (i.e., signs of operands differ and result sign mismatches).
  - Both operands are positive, result is positive, no overflow → OF = 0.
- **PF (Parity Flag)** → Set if LSB (low 8 bits) has even number of 1s.
  - Low byte = `0Eh = 00001110b` → three 1s (odd) → PF = 0.
- **AF (Auxiliary Carry Flag)** → Set if there's a borrow/carry from bit 3 to bit 4.
  - Nibble calculation: low nibble = `8h - Ah` → borrow happens → AF = 1.

- Lower nibble of 78H: 1000 (8 in decimal)
- Lower nibble of 6AH: 1010 (10 in decimal)

Now, subtract: 8 - 10 = -2. This requires a **borrow** from the higher nibble

Auxiliary Flag (AF) should be set.

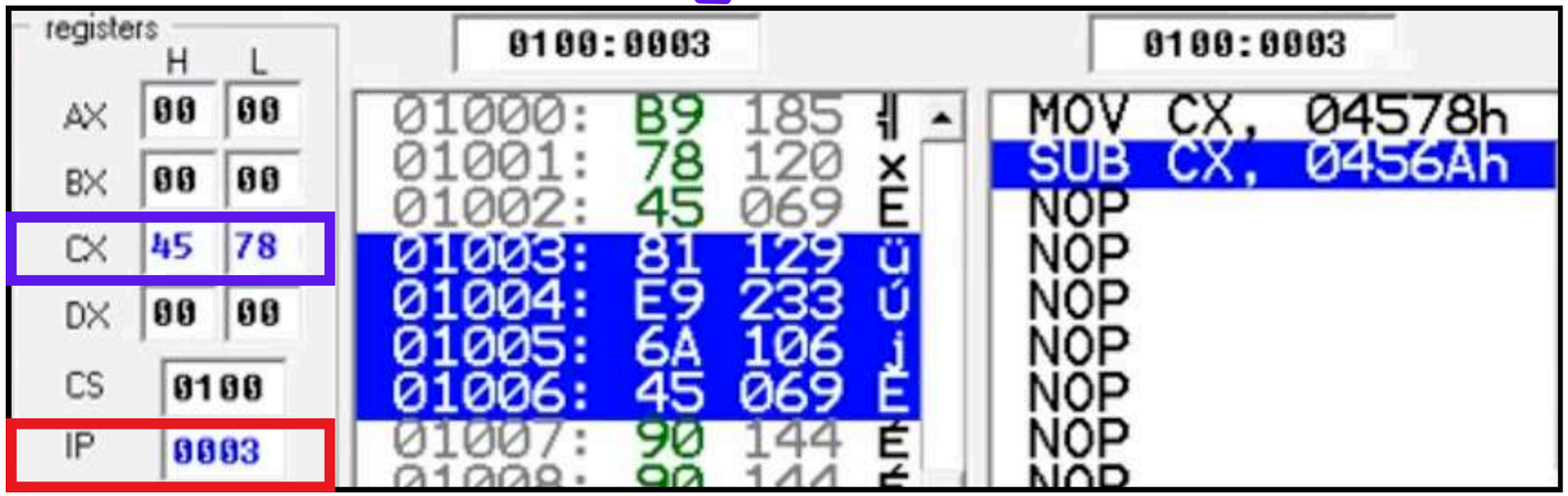
Confirm with Binary Subtraction of Low Bytes

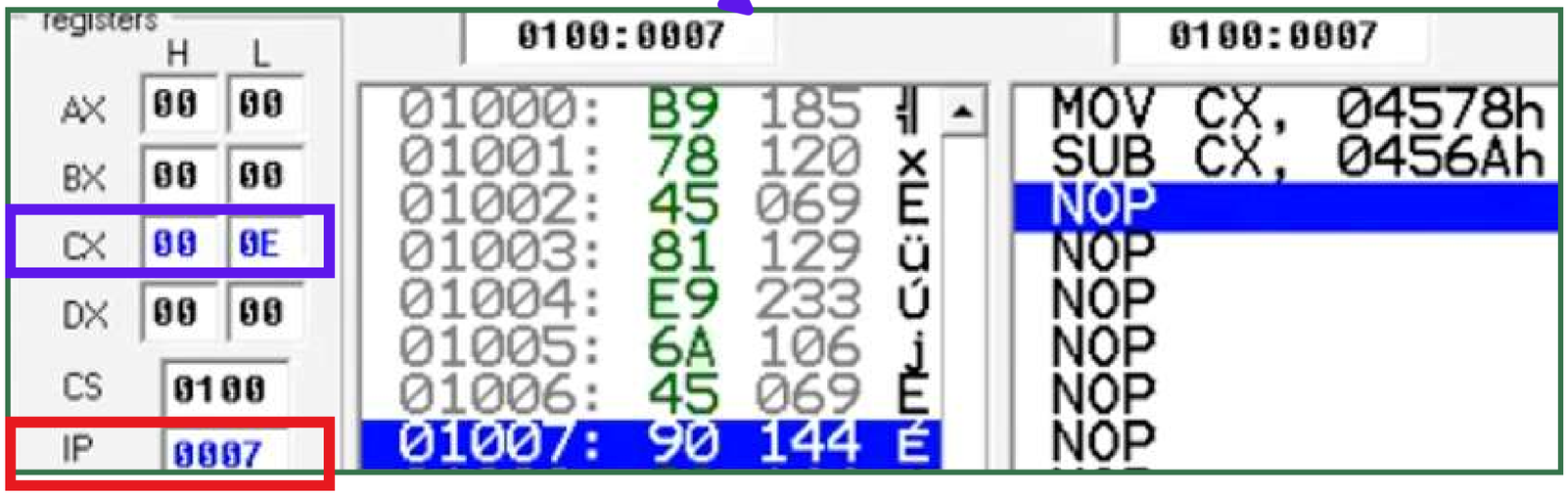
Let's subtract 6AH from 78H in binary:

```
  0111 1000   (78H)
- 0110 1010   (6AH)
---------------
  0000 1110   (0EH)
```

The **Auxiliary Flag (AF) is set (1)** because during the subtraction of the low bytes (78H - 6AH), a borrow was required from the higher nibble (bits 4-7) to the lower nibble (bits 0-3).

```
MOV CX,4578H
SUB CX,456AH
```





Using Flags for Decision Making

Flags are essential for **conditional execution**. After a CMP or arithmetic operation, conditional jump instructions use flags to control program flow.

Later in the course

Common Conditional Jumps

JE/JZ	Jump if equal/zero (ZF=1)
JNE/JNZ	Jump if not equal (ZF=0)
JG/JNLE	Jump if greater (signed)
JL/JNGE	Jump if less (signed)
JA/JNBE	Jump if above (unsigned)
JB/JNAE	Jump if below (unsigned)

Example: If-Then-Else

Compare AX and BX:

```
cmp ax, bx
```

If equal, jump:

```
je equal_label
```

Else (not equal):

```
; ... not equal code
```

```
jmp continue
```

```
equal_label:
    ; ... equal code
```

07

Memory and Addressing

UNDERSTANDING HOW DATA IS STORED AND ACCESSED

Memory Organization

Memory in the 8086 is organized as a **linear array of bytes**. Each byte has a unique address, and multi-byte data is stored in consecutive memory locations.

Key Characteristics

Byte-Addressable
Each byte has a unique address (0 to FFFFFH)

Multi-Byte Storage
Words (2 bytes) and double words (4 bytes)

Little-Endian
Low byte stored first, high byte second

8086 Memory Map

Total Addressable:	**1 MB**
Address Range:	00000H - FFFFFH
Address Bus:	20 bits
Data Bus:	16 bits

Segmented Access: 1MB is accessed using 16-bit segment and offset registers combined to form 20-bit addresses.

Displacement in Addressing

Displacement is a constant offset value stored directly in the instruction. It's used when the operand refers to a specific memory location.

How It Works

Direct Addressing

The displacement is added to the default segment (usually DS)

Example

Instruction:

MOV AX, [1234H]

Displacement:

1234H

Storage: Displacement is stored low-byte first in the instruction (34H 12H).

ModR/M Byte Explained

The **ModR/M byte** is a secondary byte that follows the opcode in many 8086 instructions. It specifies the instruction's addressing mode.

ModR/M Structure

Mod	Reg	R/M
2 bits	3 bits	3 bits

Mod: Addressing mode (register, memory, displacement)

Reg: Register operand or opcode extension

R/M: Register or memory operand

Purpose

Encode Operands

Specifies up to two operands (register and memory/register)

Addressing Mode

Determines how to interpret the operands

Compact Encoding

Single byte encodes complex addressing information

Example: ADD BX, AX uses ModR/M to specify BX as destination and AX as source.

Little Endian Storage

The 8086 uses little-endian byte ordering, where the **low byte is stored first** (at the lower address), followed by the high byte.

Why Little-Endian?

Natural for Math
Easier for multi-precision arithmetic

Hardware Efficiency
Simplifies CPU data access logic

Intel Tradition
Continued in x86 architecture

Example: Storing 1234H

Value to store:

1234H
High: 12H, Low: 34H

Memory Layout:

Address N+1	Address N
12H	34H
High Byte	Low Byte

Remember: In little-endian, read the bytes in reverse order to get the actual value!

📄 MOV Instructions (8086)

Instruction Form	Type Description	Example	Size in Bytes
`MOV r16, imm16`	Move immediate to 16-bit register	`MOV CX, 4578H`	1 (opcode) + 2 (imm) = **3**
`MOV r8, imm8`	Move immediate to 8-bit register	`MOV AH, 0CH`	1 (opcode) + 1 (imm) = **2**
`MOV r16, r16` / `r8,r8`	Move register to register	`MOV BX, CX`	1 (opcode) + 1 (modR/M) = **2**
`MOV r16, [mem]`	Move from memory (direct)	`MOV AX, [1234H]`	1 (opcode) + 2 (addr) = **3**
`MOV [mem], r16`	Move register to memory (direct)	`MOV [1234H], DX`	1 (opcode) + 2 (addr) = **3**

Instruction Size: MOV Examples

The 8086 uses **variable-length instructions**. The size depends on the addressing modes and operands used.

2 Bytes

Instruction:
MOV CX, AX

Register to register (16-bit)

- Opcode: 1 byte
- ModR/M: 1 byte

2 Bytes

Instruction:
MOV DH, 24H

Immediate to 8-bit register

- Opcode: 1 byte
- Immediate: 1 byte

3 Bytes

Instruction:
MOV AX, 1234H

Immediate to 16-bit register

- Opcode: 1 byte
- Immediate: 2 bytes

Little-Endian Storage Example

MOV AX, 1234H

Byte 0: Opcode
Byte 1: 34H (low)
Byte 2: 12H (high)

→

Memory Layout

Addr+0: Opcode
Addr+1: 34H
Addr+2: 12H

Number of bytes each one takes in 8086 assembly

- **MOV CX, AX**
  - This is move from 16-bit register to 16-bit register.
  - Total: **2 bytes**
- Line 1: 3 bytes
- Line 2: 2 bytes
- Line 3: 2 bytes
- Line 4: 2 bytes
- Total bytes = 3 + 2 + 2 + 2 = **9 bytes**

```
01   MOV AX,1234H
02   MOV DH,24H
03   MOV BL,DH
04   MOV CX,AX
```

0100 : 0009

- **MOV DH, 24H**
  - This is move immediate to 8-bit register.
  - Then the immediate value 24H is one byte.
  - Total: 1 byte opcode + 1 byte immediate = **2 bytes**.
- **MOV BL, DH**
  - This is move from 8-bit register to 8-bit register.
  - Total: 2 bytes
  - The opcode for MOV (1 byte)
  - The ModR/M byte follows the opcode and specifies the source and destination registers (1 byte)

Number of bytes each one takes in 8086 assembly

- The 8086 processor uses variable-length instructions, so the size depends on the addressing modes and operands.
- **MOV AX, 1234H**
  - This is a move immediate to 16-bit register.
  - Then the immediate value 1234H is stored as two bytes:
    - low byte first (34h) then high byte (12h).
    - So total: 1 byte for opcode **+** 2 bytes for immediate = **3 bytes.**

This is a **special short form** of the instruction that explicitly uses the **AX register**

Instruction Form	Type Description	Example	Size in Bytes
`ADD AX, imm16`	Add immediate to AX (short form)	`ADD AX, 1234H`	1 (opcode) + 2 (imm) = **3**
`ADD AL, imm8`	Add immediate to AL (short form)	`ADD AL, 12H`	1 (opcode) + 1 (imm) = **2**
`ADD r16, imm16`	Add immediate to 16-bit register	`ADD BX, 5678H`	1 (opcode) + 1 (modR/M) + 2 (imm) = **4**
`ADD r8, imm8`	Add immediate to 8-bit register	`ADD CL, 09H`	1 (opcode) + 1 (modR/M) + 1 (imm) = **3**
`ADD r16, r16` / `r8,r8`	Add register to register	`ADD DL, CL`	1 (opcode) + 1 (modR/M) = **2**

This is the **general form** for adding an immediate value to any **16-bit register: BX, CX, DX, ...**

Instruction Size: ADD Examples

2 Bytes

Instruction:
ADD BX, AX

Register to register (16-bit)

- Opcode: 1 byte
- ModR/M: 1 byte

3 Bytes

Instruction:
ADD BH, 24H

Immediate to 8-bit register

- Opcode: 1 byte
- ModR/M: 1 byte
- Immediate: 1 byte

3 Bytes

Instruction:
ADD AX, 1234H

Immediate to AX (special form)

- Opcode: 1 byte
- Immediate: 2 bytes

4 Bytes

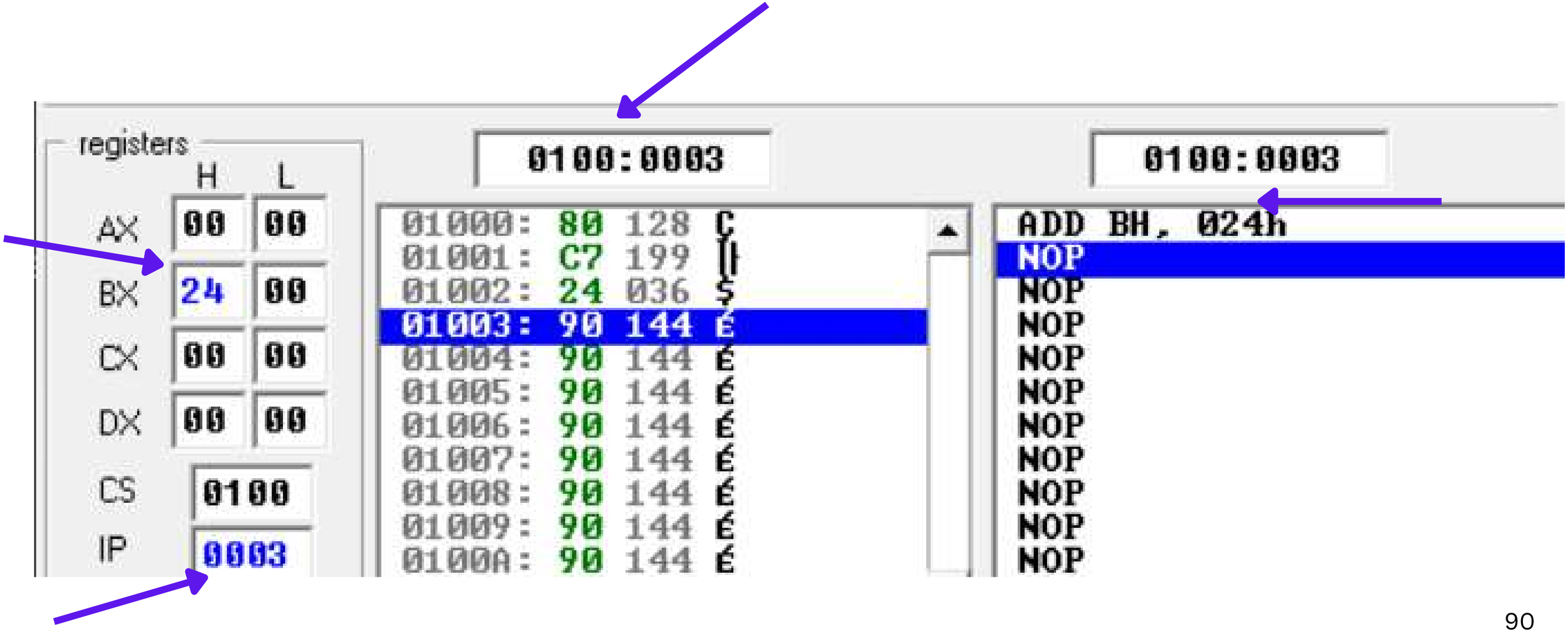
Instruction:
ADD BX, 5678H

Immediate to 16-bit register

- Opcode: 1 byte
- ModR/M: 1 byte
- Immediate: 2 bytes

Memory Breakdown

```
ADD BH,24H
```



Memory Breakdown

```
ADD BH, 24H
```

- ADD BH, 24H adds an 8-bit immediate value (24H) to an 8-bit register (BH)
- This is a case of ADD r8, imm8
- Opcode: **1 byte**
- ModR/M: **1 byte** (specifying BH as the destination)
- Immediate: **1 byte** (24H)
- Total: **3 bytes**

Memory Breakdown

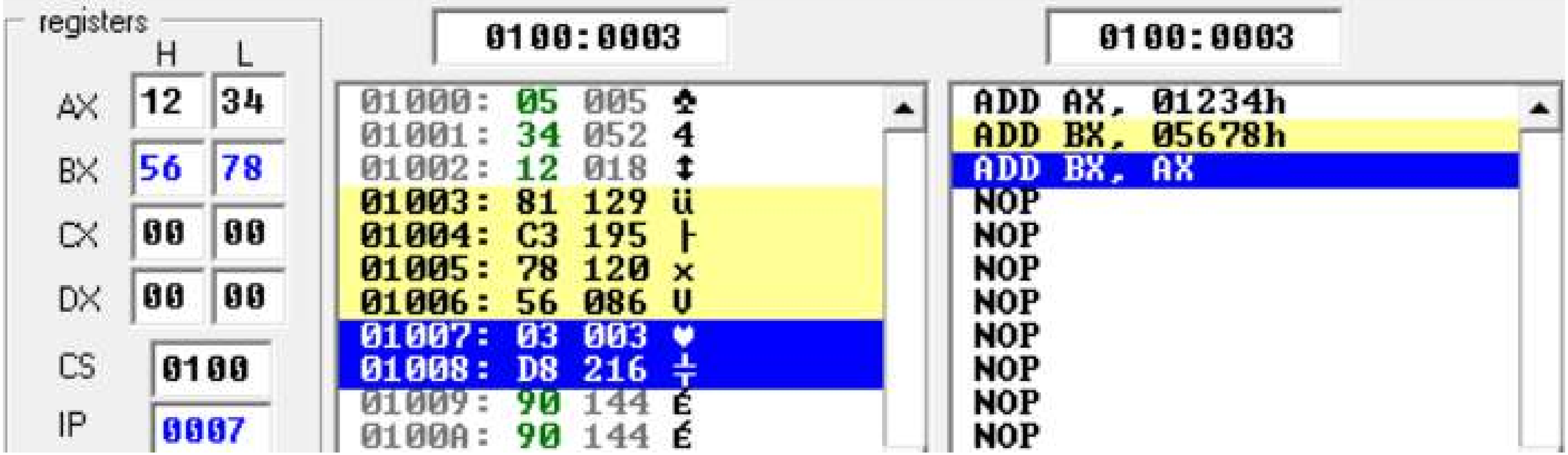
```
ADD AX, 1234H
ADD BX, 5678H
ADD BX,AX
```

```
ADD AX, 1234H
ADD BX, 5678H
ADD BX,AX
```

- **ADD AX, 1234H** adds a 16-bit immediate value (1234H) to the 16-bit register AX
- This is a case of **ADD AX, imm16**
- Opcode: **1 byte**
- Immediate: **2 byte** (1234H)
- Total: **3 bytes**

```
ADD AX, 1234H
ADD BX, 5678H
ADD BX, AX
```

Memory Breakdown

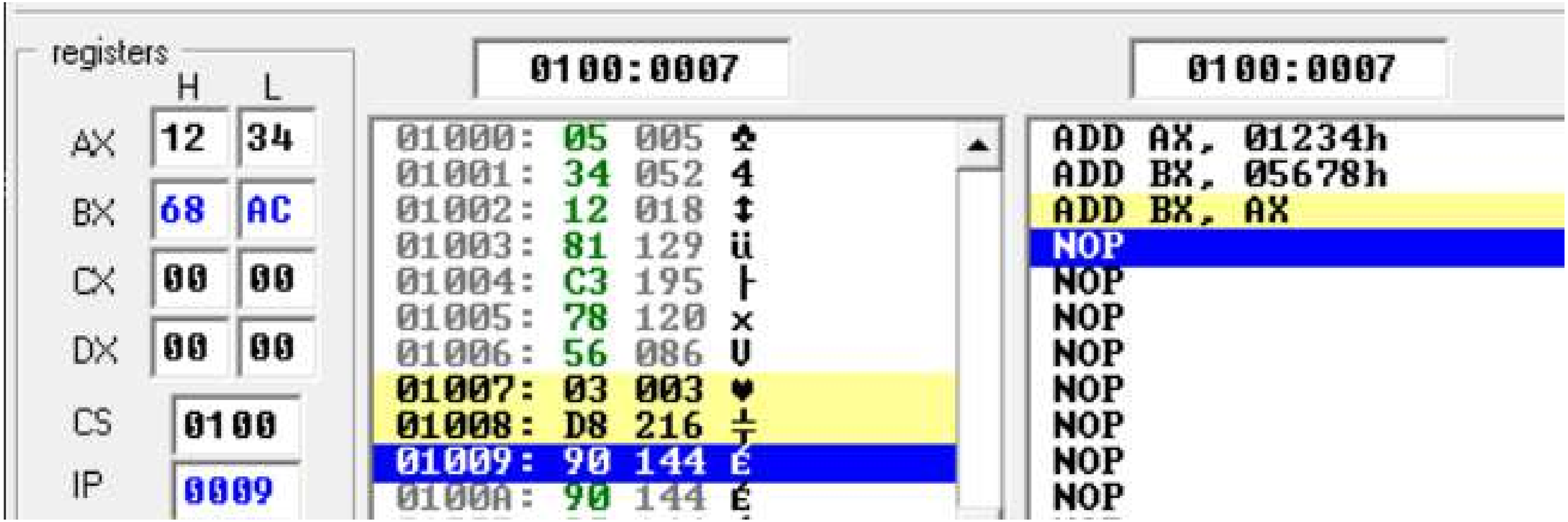


```
ADD AX, 1234H
ADD BX, 5678H
ADD BX,AX
```

- **ADD BX, 5678H** adds a 16-bit immediate value (5678H) into the 16-bit register BX
- This is a case of **ADD r16, imm16**
- Opcode: **1 byte**
- ModR/M: specifies the register (**1 byte**)
- Immediate: **2 byte** (5678H)
- Total: **4 bytes**

```
ADD AX, 1234H
ADD BX, 5678H
ADD BX, AX
```

Memory Breakdown



```
ADD AX, 1234H
ADD BX, 5678H
ADD BX,AX
```

- **ADD BX, AX** adds the contents of the 16-bit register AX to the 16-bit register BX
- This is a case of **ADD r16, r16**
- Opcode: **1 byte**
- ModR/M: specifies the register (**1 byte**)
- Total: **2 bytes**

📄 SUB Instructions (8086)

Instruction Form	Type Description	Example	Size in Bytes
`SUB r16, imm16`	Subtract immediate from 16-bit register	`SUB CX, 456AH`	1 (opcode) + 1 (modR/M) + 2 (imm) = **4**
`SUB r8, imm8`	Subtract immediate from 8-bit register	`SUB AL, 05H`	1 (opcode + 1 (imm) = **2**
`SUB r16, r16` / `r8,r8`	Subtract register from register	`SUB AH, BL`	1 (opcode) + 1 (modR/M) = **2**

Instruction Size: SUB Examples

2 Bytes

Instruction:

MOV AH, 0CH

Immediate to 8-bit register

Opcode: 1 byte

Immediate: 1 byte

2 Bytes

Instruction:

MOV BL, 08H

Immediate to 8-bit register

Opcode: 1 byte

Immediate: 1 byte

2 Bytes

Instruction:

SUB AH, BL

Register to register (8-bit)

Opcode: 1 byte

ModR/M: 1 byte

Complete SUB Example Breakdown

Code Sequence:

MOV AH, 0CH ; 2 bytes

MOV BL, 08H ; 2 bytes

SUB AH, BL ; 2 bytes

Total Size:

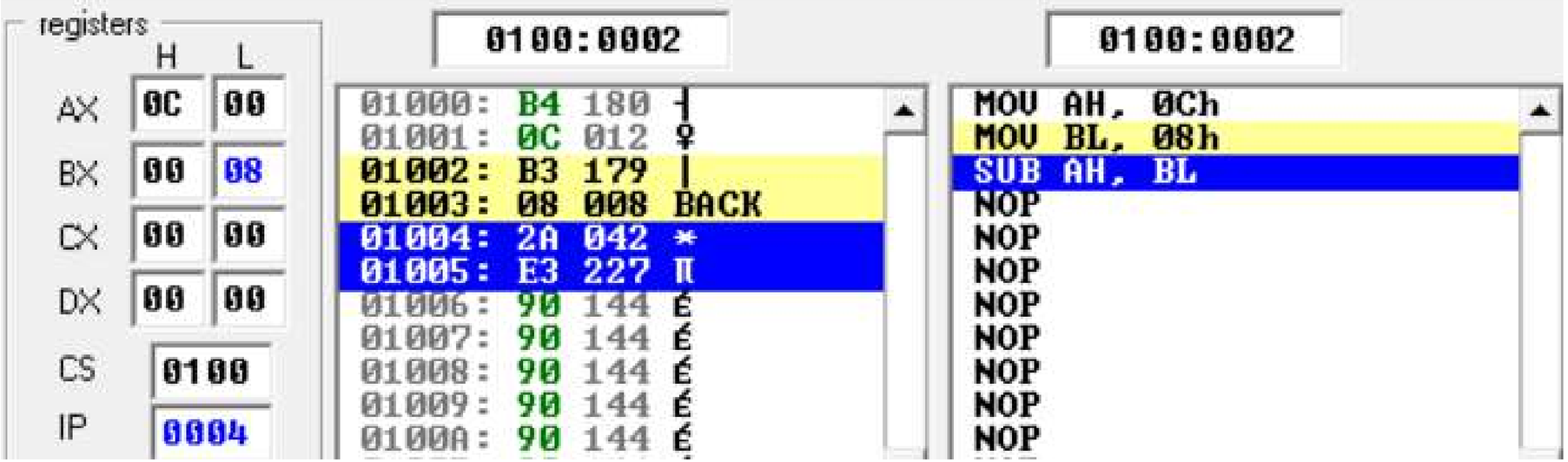
6 bytes

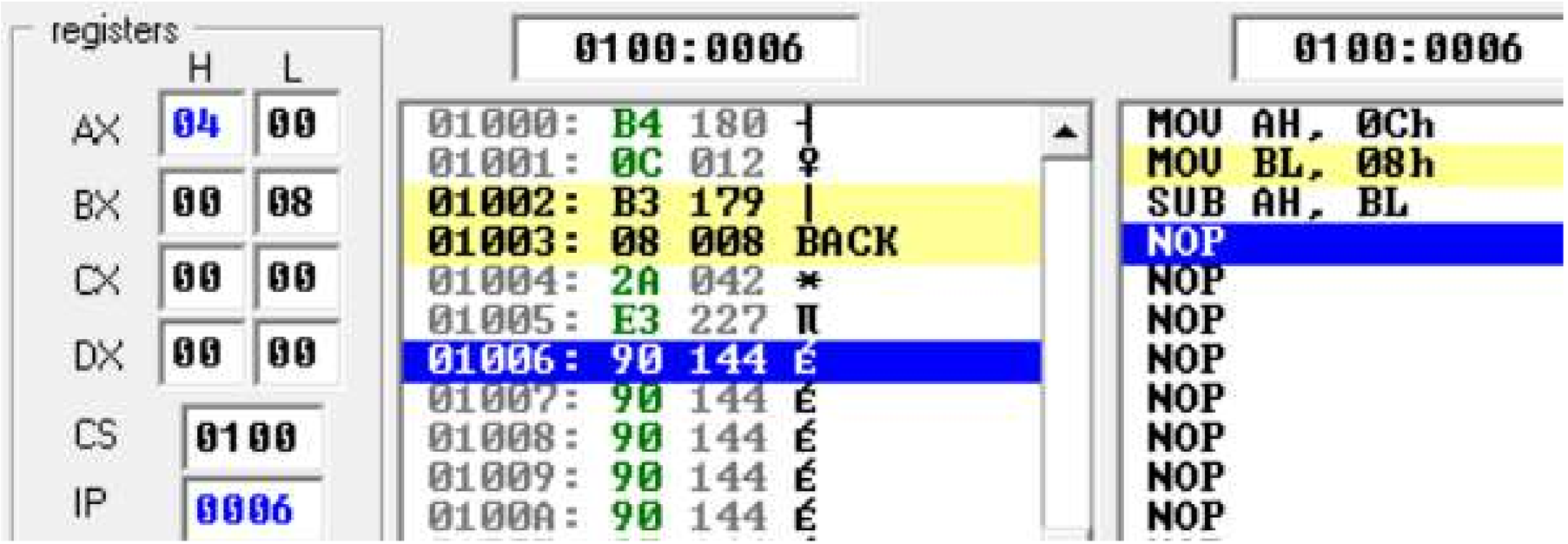
For 3 instructions

Memory Breakdown

- **MOV AH, 0CH**
  - **Total size:** 1 byte (opcode) + 1 byte (immediate) = **2 bytes**
- **MOV BL, 08H**
  - **Total size:** 1 byte (opcode) + 1 byte (immediate) = **2 bytes**
- **SUB AH, BL**
  - No immediate value, since both operands are registers
  - **Total size:** 1 byte (opcode) + 1 byte (modR/M) = **2 bytes**

- The **modR/M byte** is a special second byte that follows many x86 opcodes.
- It tells the CPU what operands the instruction should use.


registers
H L
AX 0C 00
BX 00 08
CX 00 00
DX 00 00
CS 0100
IP 0004
0100:0002
01000: B4 180 ┤
01001: 0C 012 ♀
01002: B3 179 │
01003: 08 008 BACK
01004: 2A 042 *
01005: E3 227 π
01006: 90 144 É
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
0100:0002
MOV AH, 0Ch
MOV BL, 08h
SUB AH, BL
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP

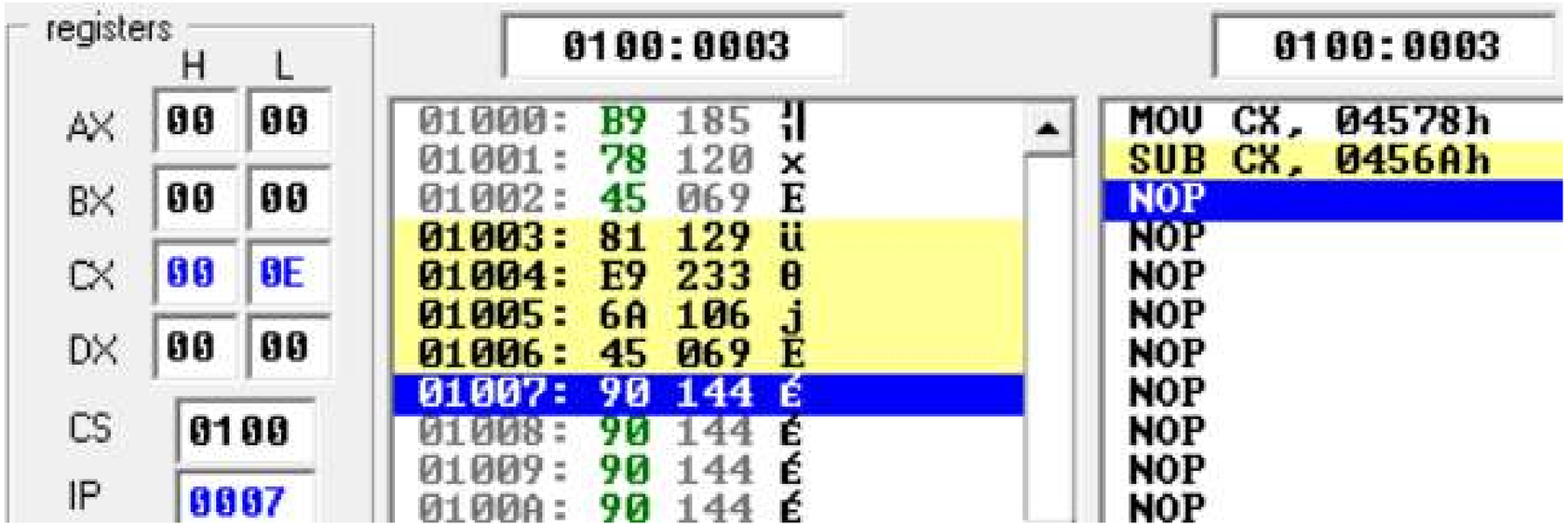

registers
H L
AX 04 00
BX 00 08
CX 00 00
DX 00 00
CS 0100
IP 0006
0100:0006
01000: B4 180 ┤
01001: 0C 012 ♀
01002: B3 179 │
01003: 08 008 BACK
01004: 2A 042 *
01005: E3 227 π
01006: 90 144 É
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
0100:0006
MOV AH, 0Ch
MOV BL, 08h
SUB AH, BL
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP

```
MOV CX,4578H
SUB CX,456AH
```

Memory Breakdown

- **MOV CX, 4578H**
  - **Instruction type:** Move immediate to 16-bit register.
  - **Immediate value:** 4578H stored low byte first → 78h then 45h.
  - **Total size:** 1 byte (opcode) + 2 bytes (immediate) = **3 bytes.**
- **SUB CX, 456AH**
  - **Instruction type:** Subtract immediate 16-bit from register.
  - **Immediate value:** 456AH stored low byte first → 6Ah then 45h.
  - **Total size:** 1 byte (opcode) + 1 byte mode R/M + 2 bytes (immediate) = **4 bytes**.

registers
H L
AX 00 00
BX 00 00
CX 45 78
DX 00 00
CS 0100
IP 0003
0100:0000
01000: B9 185 ¦
01001: 78 120 x
01002: 45 069 E
01003: 81 129 ü
01004: E9 233 Θ
01005: 6A 106 j
01006: 45 069 E
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
0100:0003
MOV CX, 04578h
SUB CX, 0456Ah
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP


registers
H L
AX 00 00
BX 00 00
CX 00 0E
DX 00 00
CS 0100
IP 0007
0100:0003
01000: B9 185 ¦
01001: 78 120 x
01002: 45 069 E
01003: 81 129 ü
01004: E9 233 Θ
01005: 6A 106 j
01006: 45 069 E
01007: 90 144 É
01008: 90 144 É
01009: 90 144 É
0100A: 90 144 É
0100:0003
MOV CX, 04578h
SUB CX, 0456Ah
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP

```
MOV AX, 456AH    ; 3 Bytes 1 Opcode + 2 Immediate Value
MOV CX, 456AH    ; 3 Bytes 1 Opcode + 2 Immediate Value
SUB AX, 456AH    ; 3 Bytes 1 Opcode + 2 Immediate Value
ADD AX, 456AH    ; 3 Bytes 1 Opcode + 2 Immediate Value
SUB CX, 456AH    ; 4 Bytes 1 Opcode + 1 Mod R/M + 2 Immediate Value
ADD CX, 456AH    ; 4 Bytes 1 Opcode + 1 Mod R/M + 2 Immediate Value
```

```
MOV AX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
MOV CX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
SUB AX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
ADD AX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
SUB CX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
ADD CX, 0012H    ; 3 Bytes 1 Opcode + 2 Immediate Value
```

In 8-bit data (e.g., 0023H with leading zeros), the **MOD R/M byte isn't needed for BX, CX, and DX** due to predefined processor rules.

CONCLUSION

Key Takeaways

1 Program Structure

Assembly programs have three sections: data (constants), bss (variables), and text (code). Understanding this structure is fundamental.

2 Registers

The 8086 has general-purpose (AX, BX, CX, DX), segment (CS, DS, SS, ES), pointer (IP, SP, BP), and index (SI, DI) registers.

3 Basic Instructions

MOV copies data, ADD/SUB perform arithmetic, CMP compares values. Each affects flags differently.

4 Memory Addressing

The 8086 uses segmented memory (segment:offset) to access 1MB. Physical address = segment × 16 + offset.

5 FLAGS Register

Status flags (CF, ZF, SF, OF, PF, AF) reflect operation results. Control flags (IF, DF) affect processor behavior.

6 Practice

Use emu8086 to write, compile, and debug assembly programs. Step through code to see registers and flags change.

Continue Your Assembly Journey

ASSEMBLY LANGUAGE IS THE FOUNDATION OF COMPUTER PROGRAMMING. MASTER THESE BASICS AND YOU'LL UNDERSTAND HOW SOFTWARE TRULY INTERACTS WITH HARDWARE.

Next Steps: Practice with emu8086, write your first programs, experiment with different instructions, and build your understanding step by step. The journey to mastering assembly starts with a single instruction!

FROM BINARY TO MASTERY

THE JOURNEY CONTINUES